

7 stycznia 2026 r.

System rekomendacyjny dla platformy e-commerce z wykorzystaniem narzędzi Apache i Google Cloud Platform

Zespół KGW Gawron

Natalia Choszczyk, Filip Langiewicz, Mikołaj Rowicki



Opis projektu

- System analityczny dla platform e-commerce
- Łączy przetwarzanie strumieniowe w czasie rzeczywistym z analizą wsadową
- Generowanie spersonalizowanych rekomendacji produktowych
- Uruchomiony na VM w Google Cloud Platform

Cel biznesowy

- Personalizowane rekomendacje na bazie profilu demograficznego
- Analiza lejka zakupowego – identyfikacja punktów krytycznych
- Monitoring trendów w czasie rzeczywistym
- Optymalizacja procesu sprzedaży i współczynników konwersji

Platformy e-commerce muszą:

- **Reagować w czasie rzeczywistym** – monitorować trendy i preferencje klientów
- **Personalizować** – dostosowywać ofertę do profilu każdego użytkownika
- **Optymalizować** – identyfikować miejsca, gdzie klienci porzucają zakupy
- **Skalować** – obsłużyć miliony użytkowników i dziesiątki tysięcy produktów

Nasze rozwiązanie

Architektura Lambda:

1. **Speed Layer** – analiza w czasie rzeczywistym
2. **Batch Layer** – głębokie analizy historyczne
3. **Serving Layer** – błyskawiczny dostęp do wyników

Źródła danych

- Strumień aktywności użytkowników: view, cart, purchase – Kaggle 9.2 GB
- Katalog produktów: ~205 tys. produktów, 1162 kategorie
- Użytkownicy: 5 mln rekordów z danymi demograficznymi

Strumień aktywności użytkowników

event_time	event_type	product_id	category_id	category_code	brand	price	user_id	user_session
2019-10-01 00:00:00 UTC	view	44600062	2103807459 595387724	NaN	shiseido	35.79	5413121 40	72d76fde-8bb3-4e00 -8c23-a032dfed738c
2019-10-01 00:00:00 UTC	view	3900821	2053013552 326770905	appliances.environment. water_heater	aqua	33.20	5547487 17	9333dfbd-b87a-4708 -9857-6336556b0fcc
2019-10-01 00:00:01 UTC	view	17200506	2053013559 792632471	furniture.living_room.sofa	NaN	543.1 0	5191072 50	566511c2-e2e3-422b -b695-cf8e6e792ca8
2019-10-01 00:00:01 UTC	view	1307067	2053013558 920217191	computers.notebook	lenovo	251.7 4	5500508 54	7c90fc70-0e80-4590 -96f3-13c02c18c713
2019-10-01 00:00:04 UTC	view	1004237	2053013555 631882655	electronics.smartphone	apple	1081. 98	5358712 17	c6bd7419-2748-4c56 -95b4-8cec9ff8b80d

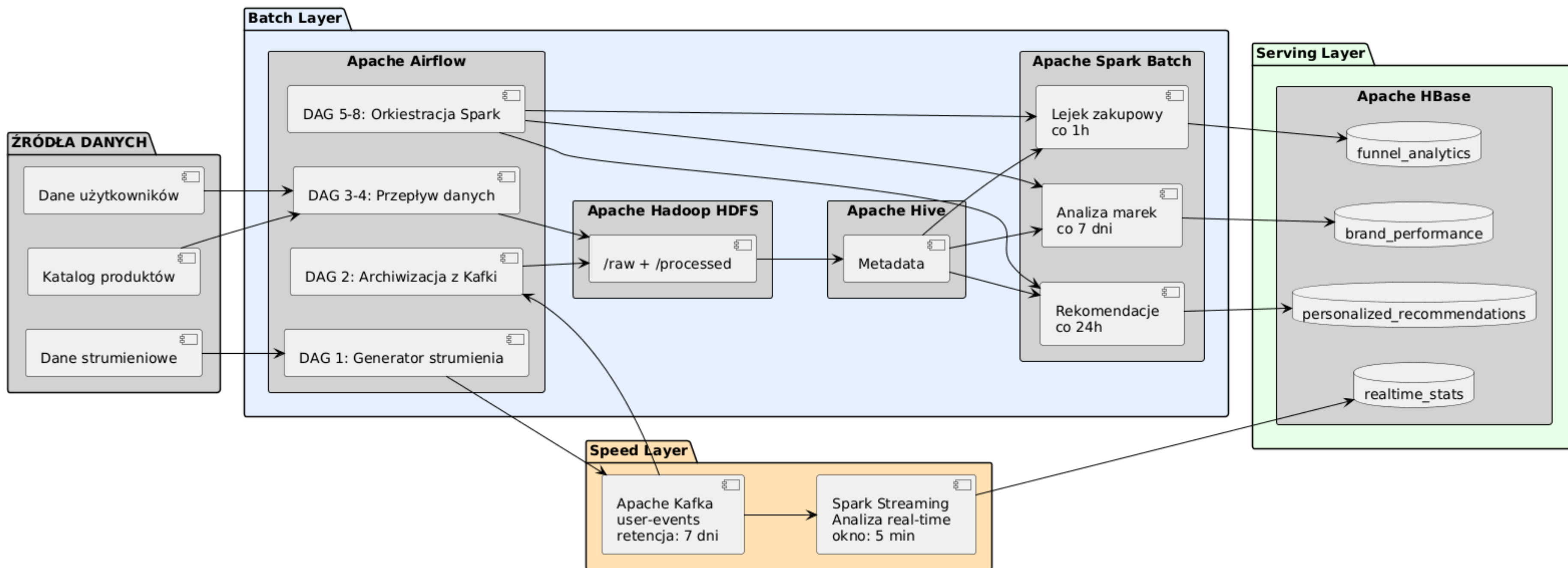
Katalog produktów

product_id	product_name	category_id	category_name	category_code	brand
1005105	work lights Model 1178	223273209307 7520756	work lights	construction.tools.light	apple
22700068	fishing equipment Model 101268	223273209164 3068746	fishing equipment		force

Dane użytkowników

user_id	name	surname	birth_date	city	zip_code	street_name	house_number
29515875	LENA	WITKOWSKA	1987-02-08	Oława	55-201	ul. Elizy Orzeszkowej	69/132
29990697	MARTYNA	WROŃSKA	1974-05-26	Rzeszów	35-202	ul. Leona Staniszewskiego	68
30493659	KINGA	DUDEK	1973-02-19	Warszawa	05-075	ul. Malinowa	50
31198833	PAWEŁ	SZYMCZAK	1999-06-30	Warszawa	02-937	ul. Rumiankowa	31/149
32836036	DAWID	JANIK	1976-11-28	Rzeszów	35-111	ul. Poprzeczna	23
34916060	IRENEUSZ	KAMIŃSKI	1975-10-19	Warszawa	00-177	ul. Błękitna	43

Architektura systemu



Wykorzystane narzędzia i ich rola

Apache Airflow 2.8.1

Centralny orkiestrator – automatyzuje i monitoruje wszystkie procesy

- DAG-i: symulacja strumienia, archiwizacja danych, analizy
- Harmonogramy: co godzinę, codziennie, tygodniowo
- Odporność: wznowienie po awarii, retry mechanizmy
- Monitoring: WebUI na porcie 8080 – wizualizacja i logi

Apache Kafka 3.6.1 – buforowanie danych

Topik user-events: architektura i funkcje

- **Buforowanie:** Przechowuje zdarzenia przez 14 dni, zanim je usunie (polityka retencji czasowej)
- **3 partycje:** Pozwalają na równoległe przetwarzanie każdy consumer czyta inną partycję
- **Partycjonowanie po user_id:** Gwarantuje, że wszystkie zdarzenia jednego użytkownika idą do tej samej partycji (zachowanie kolejności)
- **Kompresja gzip:** Zmniejsza rozmiar o 70-80%, zmniejsza transfer sieciowy i użycie dysku
- **Separacja producenta od konsumentów:** Symulator publikuje zdarzenia; Spark Streaming, archiwizacja i analiza działają niezależnie bez blokowania się
- **Przepustowość:** ~15 zdarzeń/sekundę w symulacji, topik buforuje skoki ruchu

Apache Hadoop 3.3.6 (HDFS)

Rozproszony system plików do archiwizacji i długoterminowego przechowywania

- **/raw** – surowe zdarzenia z Kafki (godzinowe archiwizacje), katalog produktów, profile użytkowników
- **/processed** – wyniki analiz Spark (funnel, recommendations, brand_performance, realtime_stats)
- **Format:** Parquet + kompresja Snappy (~60% redukcji rozmiaru)
- **Partycjonowanie:** rok/miesiąc/dzień/godzina dla szybkiego filtrowania czasowego
- **Pojemność:** 194 GB na VM, automatyczna replikacja wzmacniająca odporność na awarie

Apache Hive 3.1.3 – hurtownia danych

Warstwa SQL nad HDFS – ujednolicony dostęp do wszystkich danych

- **Tabela user_events** – 67.5 mln zdarzeń, partycjonowana po roku/miesiącu/dniu /godzinie, format Parquet
- **Tabela product_catalog** – 205 tys. produktów, partycjonowana po category_code
- **Tabela user_profiles** – 5 mln użytkowników z obliczonymi grupami wiekowymi (18-24, 25-34, itd.)
- **Metastore:** Derby DB przechowuje schemat tabel, typy kolumn, lokalizacje w HDFS
- **Integracja ze Spark:** Spark SQL czyta metadane z Hive i operuje bezpośrednio na plikach Parquet (szybciej niż native Hive)

Apache Spark 3.5.3 – silnik analityczny

Jednocześnie przetwarzanie strumieniowe (real-time) i wsadowe (batch)

- **Speed Layer:** Spark Structured Streaming konsumuje dane z Kafki w 5-minutowych oknach czasowych
- **Batch Layer:** Zadania codzienne/tygodniowe czytają całą historię z Hive dla głębokich analiz
- **Zasoby:** Tryb standalone, 4 vCPU, 4GB RAM driver, 2GB RAM executory
- **Checkpoint mechanism:** Gwarantuje dokładnie jednokrotne przetworzenie każdego zdarzenia, automatyczne wznowienie po awarii
- **Zapis wyników:** Wyniki trafiają do HBase (dla szybkiego dostępu) i HDFS/Parquet (dla archiwum)

Apache HBase 2.6.4 – warstwa serwisowa

Dostęp do wstępnie obliczonych wyników:

- **personalized_recommendations** – rekomendacje dla każdego segmentu demograficznego (top 10 produktów)
- **funnel_analytics** – metryki lejka (view → cart → purchase) per kategoria/marka z drop-off rates
- **realtime_stats** – 5 analiz w 5-minutowych oknach (aktywni użytkownicy, top produkty, top marki, metryki, rozkład zdarzeń)
- **brand_performance** – tygodniowe analizy wydajności marek (konwersja, przychód, segmentacja demograficzna)
- **Thrift Server (port 9090)**: Umożliwia dostęp z PySpark, automatyczne wznowienie przez systemd
- **Przechowywanie**: Dane w HDFS z replikacją, model klucz-wartość z wersjonowaniem zmian

Hive Metastore

Hive przechowuje metadane; Spark SQL czyta poprzez Hive Metastore

- **Tabela user_events:** EXTERNAL (wskazuje na HDFS), partycjonowana po (year, month, day, hour)
- **Metadane przechowywane w:** Derby DB (baza SQLite) w HDFS; zawiera schemat, typy, lokalizacje, partycje
- **Spark SQL:** `SELECT * FROM user_events WHERE year=2025 AND month=12` → Spark czyta metadane z Hive → otwiera pliki Parquet w HDFS (bez użycia Hive executora)
- **Optymalizacja:** Spark SQL bierze lokalizacje partycji z Hive, szybciej niż native Hive, bo omija Hive translator
- **Partycje:** Każda godzina ma swoją partycję; zapytanie za ostatnie 24h czyta 24 partycje, pomija starsze
- **Skala:** 67.5 mln rekordów zdarzeń + 5 mln profili użytkowników + 205k produktów = ~80-100 GB raw data w Parquet

Środowisko wdrożeniowe

- **VM:** bigdata-vm, n1-standard-4 (4 vCPU @ 2.2 GHz, 14 GB RAM, 194 GB dysk)
- **OS:** Ubuntu 22.04.5 LTS, Java 17 OpenJDK (wymagane dla Apache)
- **Sieci:** Airflow UI (8080), HDFS NameNode (9870), HBase Master (16010), SSH (22)
- **Bezpieczeństwo:** Firewall GCP ogranicza dostęp do portów wewnętrznych Kafki (9092), HBase Thrift (9090), Zookeeper (2181)

Przetwarzanie strumieniowe

Airflow DAG – realtime_ecommerce_stream

Sekwencja 6 zadań, uruchamianych codziennie o północy

1. **check_date_eligibility** – weryfikacja, czy bieżąca data kwalifikuje się do symulacji
2. **check_kafka_health** – sprawdzenie, czy Kafka jest uruchomiona, automatyczne uruchomienie, jeśli potrzeba
3. **cleanup_old_simulators** – zabicie zombie procesów z poprzednich dni, czyszczenie starych plików PID
4. **ensure_kafka_topic** – stworzenie topiku user-events (3 partycje, 14-dniowa retencja, kompresja gzip)
5. **run_realtime_simulator** – uruchomienie symulatora w tle (nohup), sprawdzenie, czy już nie działa dla tej daty
6. **initial_monitor** – weryfikacja, czy symulator się uruchomił poprawnie i zdarzenia trafiają do Kafki



```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ tail -50 /home/win10/airflow/logs/kafka_realtime_simulator.log
2025-12-30 16:57:59,002 - INFO - Postę: wysłano 2007000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:58:24,003 - INFO - Postę: wysłano 2008000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:58:50,009 - INFO - Postę: wysłano 2009000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:59:16,003 - INFO - Postę: wysłano 2010000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:59:42,004 - INFO - Postę: wysłano 2011000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:07,004 - INFO - Postę: wysłano 2012000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:33,003 - INFO - Postę: wysłano 2013000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:58,005 - INFO - Postę: wysłano 2014000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:01:23,004 - INFO - Postę: wysłano 2015000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:01:47,011 - INFO - Postę: wysłano 2016000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:02:14,006 - INFO - Postę: wysłano 2017000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:02:39,011 - INFO - Postę: wysłano 2018000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:03,002 - INFO - Postę: wysłano 2019000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:28,008 - INFO - Postę: wysłano 2020000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:54,002 - INFO - Postę: wysłano 2021000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:04:20,003 - INFO - Postę: wysłano 2022000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:04:46,007 - INFO - Postę: wysłano 2023000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:05:12,002 - INFO - Postę: wysłano 2024000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:05:38,002 - INFO - Postę: wysłano 2025000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:06:03,004 - INFO - Postę: wysłano 2026000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:06:29,004 - INFO - Postę: wysłano 2027000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:06:55,004 - INFO - Postę: wysłano 2028000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:07:21,004 - INFO - Postę: wysłano 2029000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:07:47,004 - INFO - Postę: wysłano 2030000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:08:12,012 - INFO - Postę: wysłano 2031000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:08:38,007 - INFO - Postę: wysłano 2032000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:04,011 - INFO - Postę: wysłano 2033000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:29,007 - INFO - Postę: wysłano 2034000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:55,003 - INFO - Postę: wysłano 2035000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:10:22,008 - INFO - Postę: wysłano 2036000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:10:48,009 - INFO - Postę: wysłano 2037000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:11:14,002 - INFO - Postę: wysłano 2038000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:11:39,006 - INFO - Postę: wysłano 2039000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:06,001 - INFO - Postę: wysłano 2040000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:33,001 - INFO - Postę: wysłano 2041000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:58,004 - INFO - Postę: wysłano 2042000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:13:23,005 - INFO - Postę: wysłano 2043000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:13:49,004 - INFO - Postę: wysłano 2044000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:14:14,002 - INFO - Postę: wysłano 2045000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:14:39,002 - INFO - Postę: wysłano 2046000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:03,003 - INFO - Postę: wysłano 2047000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:28,006 - INFO - Postę: wysłano 2048000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:55,002 - INFO - Postę: wysłano 2049000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:16:22,003 - INFO - Postę: wysłano 2050000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
```

Logi symulatora

```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ kafka-run-class.sh kafka.tools.GetOffsetShell \
--broker-list localhost:9092 \
--topic user-events
user-events:0:2358921
user-events:1:2358723
user-events:2:2348634
```

Liczba eventów zgromadzonych w Kafce na dzień 30.12.2025 r. (godzina 19:00)

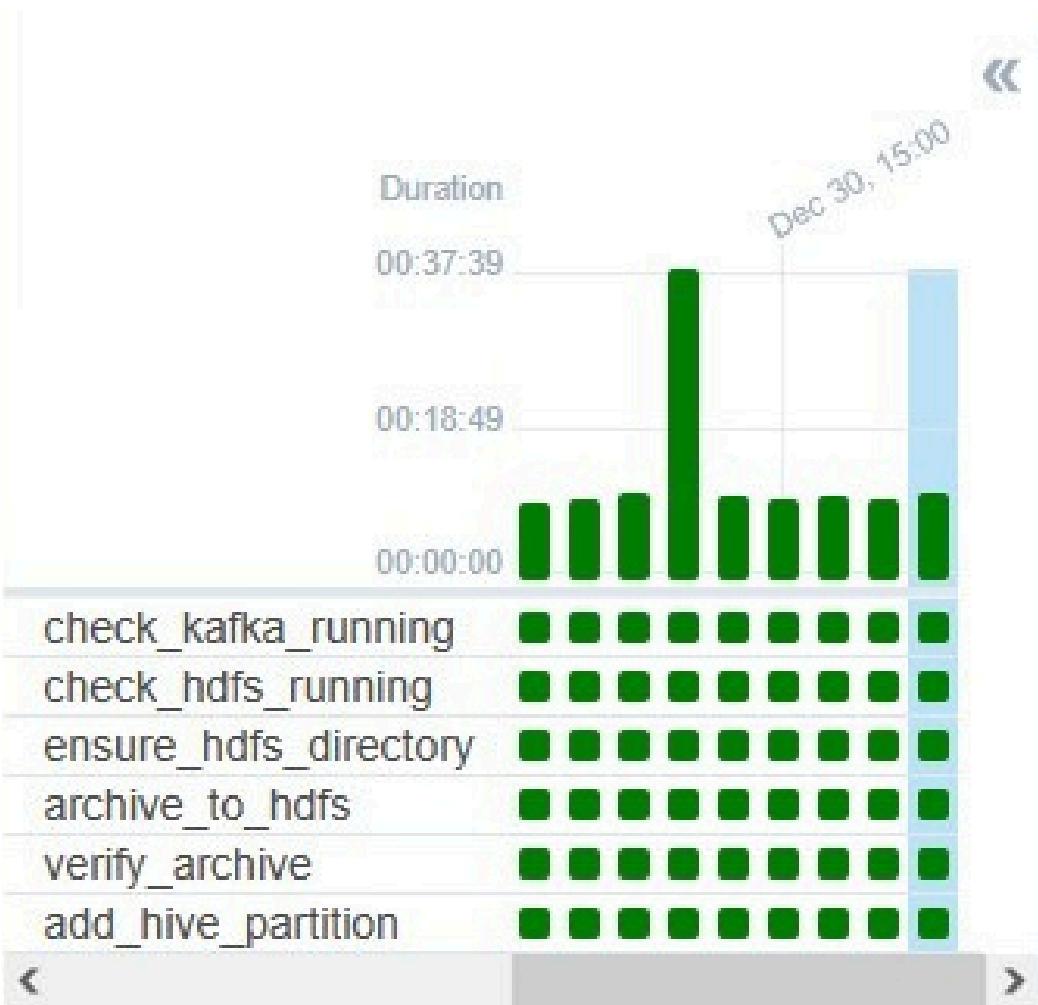
```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ timeout 5 kafka-console-consumer.sh \
--bootstrap-server localhost:9092 \
--topic user-events \
--max-messages 3 \
--from-beginning
{"event_time": "2019-12-27 00:00:01 UTC", "event_type": "view", "product_id": 6200046, "category_id": "2232732091961835858", "category_code": "appliances.environment.air_heater", "brand": "almacom", "price": 216.22, "user_id": 514029317, "user_session": "01cf396e-03f1-4ed0-ad3d-1142ad8d008f", "original_time": "2019-12-27 00:00:01 UTC", "mapped_time": "2025-12-27T00:00:01", "processing_time": "2025-12-27T00:20:24.551990"}
{"event_time": "2019-12-27 00:00:01 UTC", "event_type": "view", "product_id": 14701293, "category_id": "2053013552570040549", "category_code": "electronics.video.projector", "brand": "sv", "price": 411.59, "user_id": 529004056, "user_session": "afe756cb-50c8-4a0f-bc28-212f8ff361c5", "original_time": "2019-12-27 00:00:01 UTC", "mapped_time": "2025-12-27T00:00:01", "processing_time": "2025-12-27T00:20:24.553059"}
{"event_time": "2019-12-27 00:00:02 UTC", "event_type": "view", "product_id": 100036212, "category_id": "2053013559792632471", "category_code": "furniture.living_room.sofa", "brand": "pinskdrev", "price": 386.11, "user_id": 588256563, "user_session": "c4c488bf-3af7-4fdf-9fc8-7084721e2091", "original_time": "2019-12-27 00:00:02 UTC", "mapped_time": "2025-12-27T00:00:02", "processing_time": "2025-12-27T00:20:24.553984"}
Processed a total of 3 messages
```

Przykładowe 3 eventy zbuforowane w Kafce

Archiwizacja strumienia do HDFS

Spark job konsumuje z Kafki i zapisuje do HDFS co godzinę

- **Zadanie Airflow:** kafka_to_hdfs_archiver (co godzinę o xx:00)
- **Co robi:** Konsumuje zdarzenia z ostatniej godziny z topiku user-events (po znaczniku czasowym Kafki)
- **Transformacja:** Deserializacja JSON → Spark DataFrame ze schematem; dodanie metadanych Kafki (partition, offset)
- **Format:** Parquet + kompresja Snappy (~60% redukcji); format kolumnowy pozwala na szybkie filtrowanie
- **Partycjonowanie:** /raw/events/year=2025/month=12/day=30/hour=10; szybkie filtrowanie po dacie/godzinie w Hive
- **Automatyczna rejestracja w Hive:** Po zapisaniu Airflow dodaje nową partycję do tabeli user_events w Hive Metastore



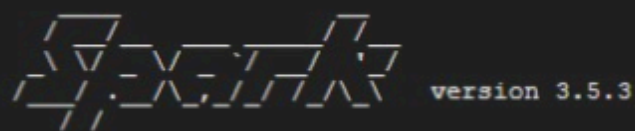
« » DAG Run
kafka_to_hdfs_archiver / 2025-12-30, 18:00:00 UTC

Clear ▾ Mark state as... ▾

△ Details Graph Gantt <> Code



```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ spark-shell --master local[2] << 'EOF'
> val df = spark.read.parquet("hdfs://localhost:9000/raw/events/year=2025/month=12/day=30/hour=10")
> println("=== SCHEMA ===")
> df.printSchema()
> println("\n=== SAMPLE DATA (5 rows) ===")
> df.show(5, truncate=false)
> println("\n=== COUNT ===")
> println(s"Total events: ${df.count()}")
> System.exit(0)
> EOF
^[[Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/30 18:51:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using bui
ltin-java classes where applicable
25/12/30 18:51:26 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
Spark context Web UI available at http://bigdata-vm.europe-central2-b.c.bigdata-e-commerce-kgw.internal:4041
Spark context available as 'sc' (master = local[2], app id = local-1767120686844).
Spark session available as 'spark'.
Welcome to
```



```
Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 1.8.0_472)
Type in expressions to have them evaluated.
Type :help for more information.
```

```
scala> val df = spark.read.parquet("hdfs://localhost:9000/raw/events/year=2025/month=12/day=30/hour=10")
df: org.apache.spark.sql.DataFrame = [event_time: string, event_type: string ... 12 more fields]
```

```
scala> println("=== SCHEMA ===")
=== SCHEMA ===
```

```
scala> df.printSchema()
root
 |-- event_time: string (nullable = true)
 |-- event_type: string (nullable = true)
 |-- product_id: long (nullable = true)
 |-- category_id: string (nullable = true)
 |-- category_code: string (nullable = true)
 |-- brand: string (nullable = true)
 |-- price: double (nullable = true)
 |-- user_id: long (nullable = true)
 |-- user_session: string (nullable = true)
 |-- original_time: string (nullable = true)
 |-- mapped_time: string (nullable = true)
 |-- processing_time: string (nullable = true)
 |-- kafka_timestamp: long (nullable = true)
 |-- kafka_partition: long (nullable = true)
```

Schemat danych zapisanych w HDFS w katalogu
/raw/events/year=2025/month=12/day=30/hour=10

```
scala> println("\n=== SAMPLE DATA (5 rows) ===")
```

```
=== SAMPLE DATA (5 rows) ===
```

```
scala> df.show(5, truncate=false)
```

```
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
|event_time      |event_type|product_id|category_id      |category_code      |brand  |price
|user_id  |user_session      |          |original_time      |mapped_time      |processing_time
|kafka_timestamp|kafka_partition|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
|2019-12-30 10:00:00 UTC|view      |38600028  |2053013557225718275|apparel.shoes.sandals      |gosh   |6.3
|514453622|4ad11043-e002-46f6-9b22-9d723d2d0cea|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00|2025-12-30T10:0
0:00.001158|17670888000001  |2          |
|2019-12-30 10:00:00 UTC|view      |19300109  |2232732109418529498|apparel.shirt              |perilla|54.03
|513211667|90daf1eb-366f-459b-a7a0-facb3cdb791f|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00|2025-12-30T10:0
0:00.002800|17670888000002  |2          |
|2019-12-30 10:00:00 UTC|view      |3300492   |2232732102414041665|computers.components.cooler|redmond|69.47
|513858781|a416b5d5-0044-4c16-be08-13427a0d8403|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00|2025-12-30T10:0
0:00.003066|17670888000003  |2          |
|2019-12-30 10:00:00 UTC|purchase   |1004708   |2232732093077520756|construction.tools.light   |huawei  |148.8
6|512562069|252f5d0b-d766-4180-b1a6-57c31e95f9d5|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00|2025-12-30T10:0
0:00.003439|17670888000003  |2          |
|2019-12-30 10:00:00 UTC|cart       |20100030  |2232732110089618156|apparel.trousers           |NULL   |28.29
|523174567|d2b2eee4-ec64-47f4-a298-2835bdf1c0dc|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00|2025-12-30T10:0
0:00.003825|17670888000003  |2          |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
only showing top 5 rows
```

```
scala> println("\n=== COUNT ===")
```

```
=== COUNT ===
```

```
scala> println(s"Total events: ${df.count()}")
Total events: 148484
```

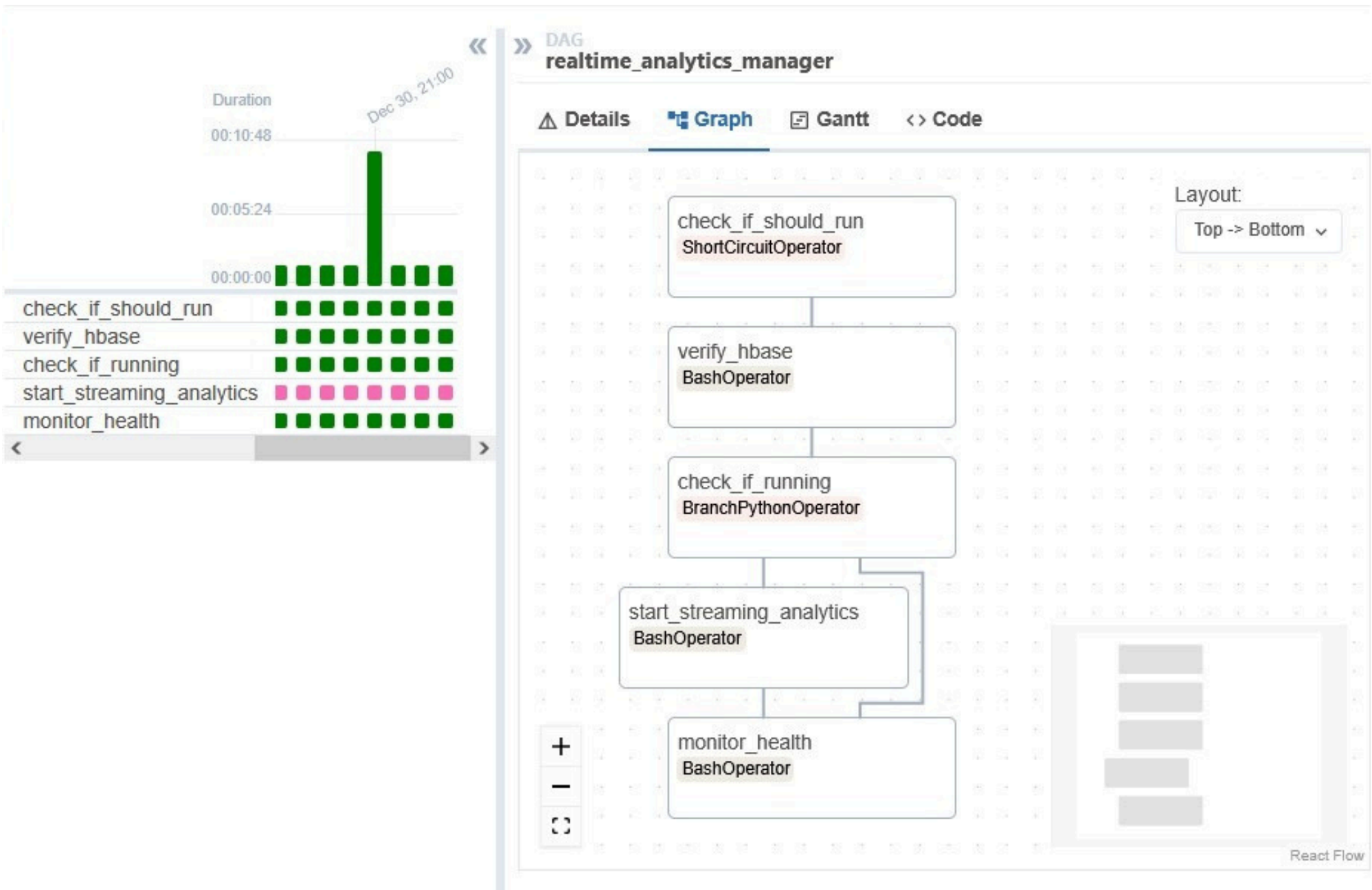
```
scala> System.exit(0)
```

5 przykładowych wierszy zapisanych w HDFS w katalogu
/raw/events/year=2025/month=12/day=30/hour=10

Pięć analiz real-time – szczegóły

Co 5 minut nowe wyniki dla każdego okna czasowego

- 1. Aktywni użytkownicy** – COUNT(DISTINCT user_id); HyperLogLog dla efektywności; pokazuje, ile osób jest aktywnych w danym momencie
- 2. Top 10 produktów (views)** – Ranking po liczbie wyświetleń; identyfikacja najpopularniejszych produktów
- 3. Metryki zakupów** – liczba dokonanych zakupów, suma osiągniętych przychodów, średnia wartość koszyka; analiza konwersji w oknie
- 4. Rozkład typów zdarzeń** – zliczanie zdarzeń typu view/cart/purchase; obliczenie wskaźników konwersji view→cart, cart→purchase
- 5. Top 10 marek** – Ranking marek po liczbie zdarzeń (view/cart/purchase); które marki generują aktywność teraz



```
bigdatacombains@bigdata-vm:~$ tail -50 /home/win10/airflow/logs/realtime_analytics.log
2025-12-30 20:24:34,993 - INFO - [EPOCH 5311] Zapisano 5 rekordów purchases do HBase
2025-12-30 20:24:34,994 - INFO - [EPOCH 6295] Zapisuję top_brands do HBase...
2025-12-30 20:24:35,146 - INFO - Received command c on object id p0
2025-12-30 20:24:35,153 - INFO - [EPOCH 5414] Zapisuję active_users do HBase...
2025-12-30 20:24:35,232 - INFO - Received command c on object id p1
2025-12-30 20:24:35,238 - INFO - [EPOCH 6245] Zapisuję top_products_views do HBase...
2025-12-30 20:24:35,592 - INFO - Received command c on object id p2
2025-12-30 20:24:35,612 - INFO - [EPOCH 5312] Zapisuję purchases do HBase...
2025-12-30 20:24:36,743 - INFO - [EPOCH 6130] Zapisano 10 rekordów event_distribution do HBase
2025-12-30 20:24:37,317 - INFO - Received command c on object id p3
2025-12-30 20:24:37,324 - INFO - [EPOCH 6131] Zapisuję event_distribution do HBase...
2025-12-30 20:24:37,886 - INFO - [EPOCH 6295] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:38,066 - INFO - [EPOCH 5414] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:38,067 - INFO - Received command c on object id p4
2025-12-30 20:24:38,074 - INFO - [EPOCH 6296] Zapisuję top_brands do HBase...
2025-12-30 20:24:38,212 - INFO - Received command c on object id p0
2025-12-30 20:24:38,218 - INFO - [EPOCH 5415] Zapisuję active_users do HBase...
2025-12-30 20:24:38,886 - INFO - [EPOCH 6245] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:39,131 - INFO - Received command c on object id p1
2025-12-30 20:24:39,137 - INFO - [EPOCH 6246] Zapisuję top_products_views do HBase...
2025-12-30 20:24:40,040 - WARNING - [EPOCH 5312] Brak danych purchases - pomijam
2025-12-30 20:24:40,521 - INFO - [EPOCH 6131] Zapisano 15 rekordów event_distribution do HBase
2025-12-30 20:24:40,637 - INFO - Received command c on object id p2
2025-12-30 20:24:40,644 - INFO - [EPOCH 5313] Zapisuję purchases do HBase...
2025-12-30 20:24:40,948 - INFO - [EPOCH 6296] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:41,440 - INFO - [EPOCH 5415] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:41,493 - INFO - Received command c on object id p3
2025-12-30 20:24:41,499 - INFO - [EPOCH 6132] Zapisuję event_distribution do HBase...
2025-12-30 20:24:41,539 - INFO - Received command c on object id p4
2025-12-30 20:24:41,546 - INFO - [EPOCH 6297] Zapisuję top_brands do HBase...
2025-12-30 20:24:42,013 - INFO - Received command c on object id p0
2025-12-30 20:24:42,022 - INFO - [EPOCH 5416] Zapisuję active_users do HBase...
2025-12-30 20:24:42,134 - INFO - [EPOCH 6246] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:42,324 - INFO - Received command c on object id p1
2025-12-30 20:24:42,330 - INFO - [EPOCH 6247] Zapisuję top_products_views do HBase...
2025-12-30 20:24:43,994 - INFO - [EPOCH 5313] Zapisano 5 rekordów purchases do HBase
2025-12-30 20:24:44,151 - INFO - [EPOCH 6132] Zapisano 15 rekordów event_distribution do HBase
2025-12-30 20:24:44,298 - INFO - Received command c on object id p3
2025-12-30 20:24:44,304 - INFO - [EPOCH 6133] Zapisuję event_distribution do HBase...
2025-12-30 20:24:44,554 - INFO - Received command c on object id p2
2025-12-30 20:24:44,562 - INFO - [EPOCH 5314] Zapisuję purchases do HBase...
2025-12-30 20:24:44,685 - INFO - [EPOCH 5416] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:44,740 - INFO - [EPOCH 6297] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:45,162 - INFO - [EPOCH 6247] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:45,237 - INFO - Received command c on object id p0
2025-12-30 20:24:45,242 - INFO - [EPOCH 5417] Zapisuję active_users do HBase...
2025-12-30 20:24:45,334 - INFO - Received command c on object id p4
2025-12-30 20:24:45,340 - INFO - [EPOCH 6298] Zapisuję top_brands do HBase...
2025-12-30 20:24:45,765 - INFO - Received command c on object id p1
2025-12-30 20:24:45,771 - INFO - [EPOCH 6248] Zapisuję top_products_views do HBase...
```

Logi procesu realtime_analytics

OKNO CZASOWE: 20191230_2111

ACTIVE USERS

- 1,142 unique users
- 3,554 total events
- 3.1 events/user avg

PURCHASES

- 56 transactions
- \$17,859.56 total revenue
- \$318.92 avg basket value
- 46 unique buyers

EVENT DISTRIBUTION

- 3,354 views
- 144 carts
- 56 purchases
- Conversion: 4.3% (view-cart)
- Conversion: 38.9% (cart-purchase)

TOP PRODUCTS (top 10 by views - sorted by count)

- Rank #01: samsung 1004767 (45 views, \$238.90)
- Rank #02: apple 1005115 (31 views, \$873.57)
- Rank #03: samsung 1005174 (20 views, \$643.23)
- Rank #04: samsung 1004873 (19 views, \$330.64)
- Rank #05: unknown 26400756 (18 views, \$325.36)
- Rank #06: apple 1002524 (17 views, \$540.04)
- Rank #07: xiaomi 1005253 (17 views, \$264.87)
- Rank #08: huawei 1004903 (17 views, \$76.94)
- Rank #09: apple 4804056 (16 views, \$164.64)
- Rank #10: apple 4804718 (16 views, \$341.58)

TOP BRANDS (top 10 by event count - sorted by count)

- Rank #01: samsung [view] (387 events, \$165,507.15)
- Rank #02: apple [view] (329 events, \$250,306.59)
- Rank #03: xiaomi [view] (235 events, \$46,298.03)

Zapis wyników do HBase

Jak Spark zapisuje wyniki streamu do HBase

- **foreachBatch:** Po każdym 5-minutowym oknie Spark wykonuje callback function, która zapisuje wyniki do HBase
- **Biblioteka happybase:** PySpark+happybase komunikują się z HBase Thrift Server (port 9090)
- **Row Key format:** timestamp|analysis_type, np. 20260102081500|active_users; pozwala szybko znaleźć dane dla konkretnego okresu
- **Column Family (CF):** Każda analiza ma swoją CF (metrics_realtime); każda metryka ma swoją kolumnę
- **Opóźnienie:** Zapisanie zajmuje ~50-100ms; wyniki dostępne w HBase praktycznie natychmiast po obliczeniu

Analiza wsadowa

Przepływ danych: użytkownicy i produkty

DAG users_batch_pipeline

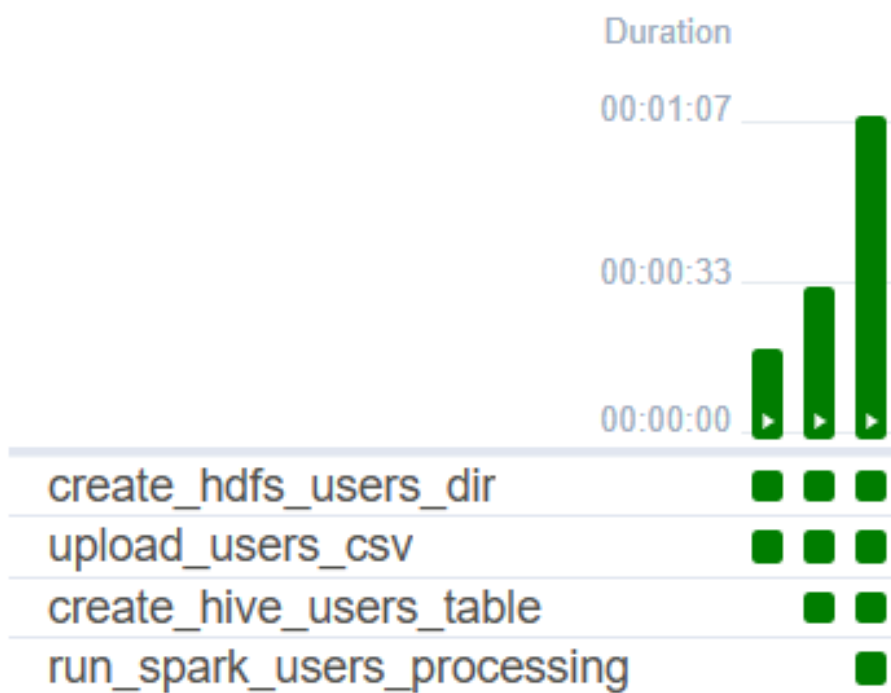
5 mln rekordów (370 MB)

1. **create_hdfs_users_dir** – katalog /raw/users
2. **upload_users_csv** – transfer z /home/win10/users.csv do HDFS
3. **create_hive_users_table** – tabela users_raw (8 kolumn: userid, name, surname, birthdate, city, zipcode, streetname, housenumber)
4. **run_spark_users_processing** – transformacje: obliczanie wieku, podział na grupy wiekowe, agregacja: miasto + grupa wiekowa

DAG products_batch_pipeline

205 tys. produktów

1. **create_hdfs_products_dir** – katalog /raw/products
2. **upload_products_csv** – transfer z /home/win10/products.csv do HDFS
3. **create_hive_products_table** – tabela products_raw (6 kolumn: productid, productname, categoryid, categoryname, categorycode, brand)
4. **run_spark_products_processing** – transformacje: usunięcie rekordów z brakującą ceną, partycjonowanie po category code



« » DAG
users_batch_pipeline

Details Graph Gantt

create_hdfs_users_dir
BashOperator

upload_users_csv
BashOperator

create_hive_users_table
BashOperator

run_spark_users_processing
BashOperator



« » DAG
products_batch_pipeline

Details Graph Gantt

create_hdfs_products_dir
BashOperator

upload_products_csv
BashOperator

create_hive_products_table
BashOperator

run_spark_products_proces...
BashOperator

Batch Layer – trzy DAG-i analityczne

Spark SQL + PySpark dla głębokich analiz historycznych

1. funnel_analysis_daily (codziennie o 03:17)

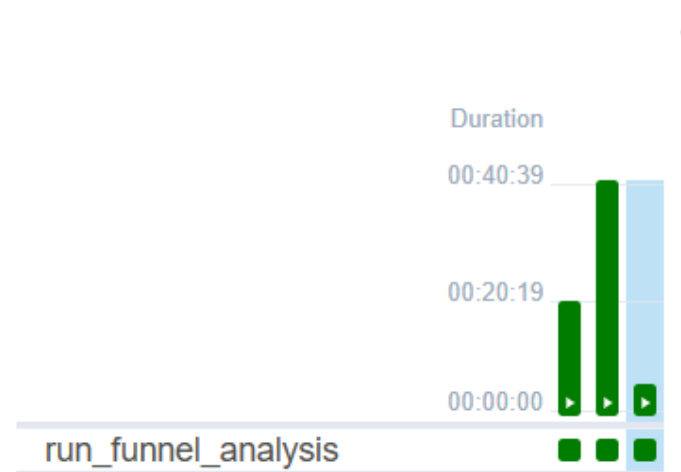
Codziennie zadanie analizuje ścieżkę view-cart-purchase na podstawie zdarzeń z ostatnich 24 godzin, oblicza wskaźniki drop-off i czasy przejść w podziale na kategorie i marki oraz zapisuje wyniki do HBase.

2. demographic_recommendations_daily (codziennie o 04:17)

Codziennie zadanie łączy zdarzenia z profilami użytkowników, segmentuje użytkowników demograficznie i oblicza wyniki rekomendacji, zapisując je do HBase.

3. brand_performance_daily (codziennie o 05:17, dane tygodniowe)

Zadanie agreguje dane zdarzeń i demograficzne, oblicza metryki efektywności marek oraz analizuje zależności z grupami wiekowymi, zapisując wyniki do HBase.



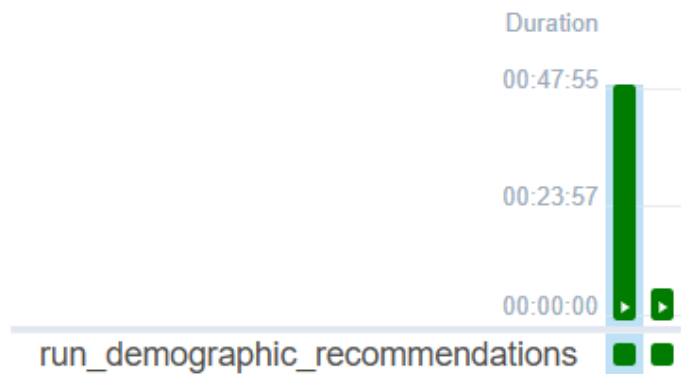
« » DAG **funnel_analysis_daily** / Run ▶ 2026-01-02, 03:17:00 UTC Clear ▾ Mark state as... ▾

[Details](#) [Graph](#) [Gantt](#) [Code](#)

DAG Run Notes:

[Add Note](#)

Status ■ success



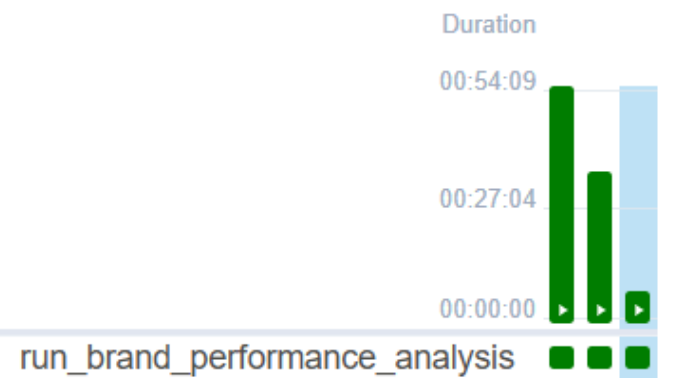
« » DAG **demographic_recommendations_daily** / Run ▶ 2026-01-02, 04:17:00 UTC Clear ▾ Mark state as... ▾

[Details](#) [Graph](#) [Gantt](#) [Code](#)

DAG Run Notes:

[Add Note](#)

Status ■ success



« » DAG **brand_performance_daily** / Run ▶ 2026-01-02, 05:17:00 UTC Clear ▾ Mark state as... ▾

[Details](#) [Graph](#) [Gantt](#) [Code](#)

DAG Run Notes:

[Add Note](#)

Status ■ success

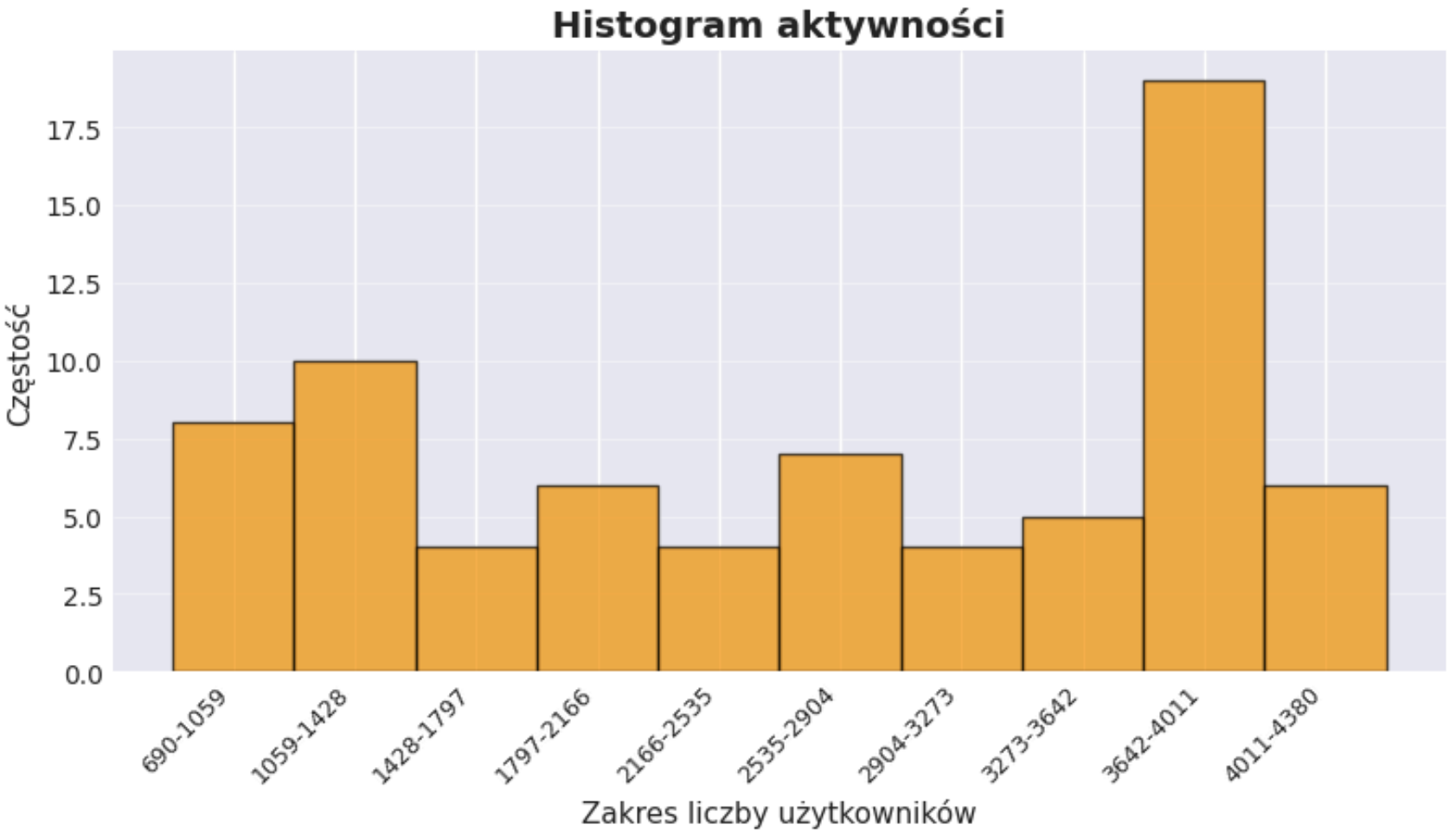
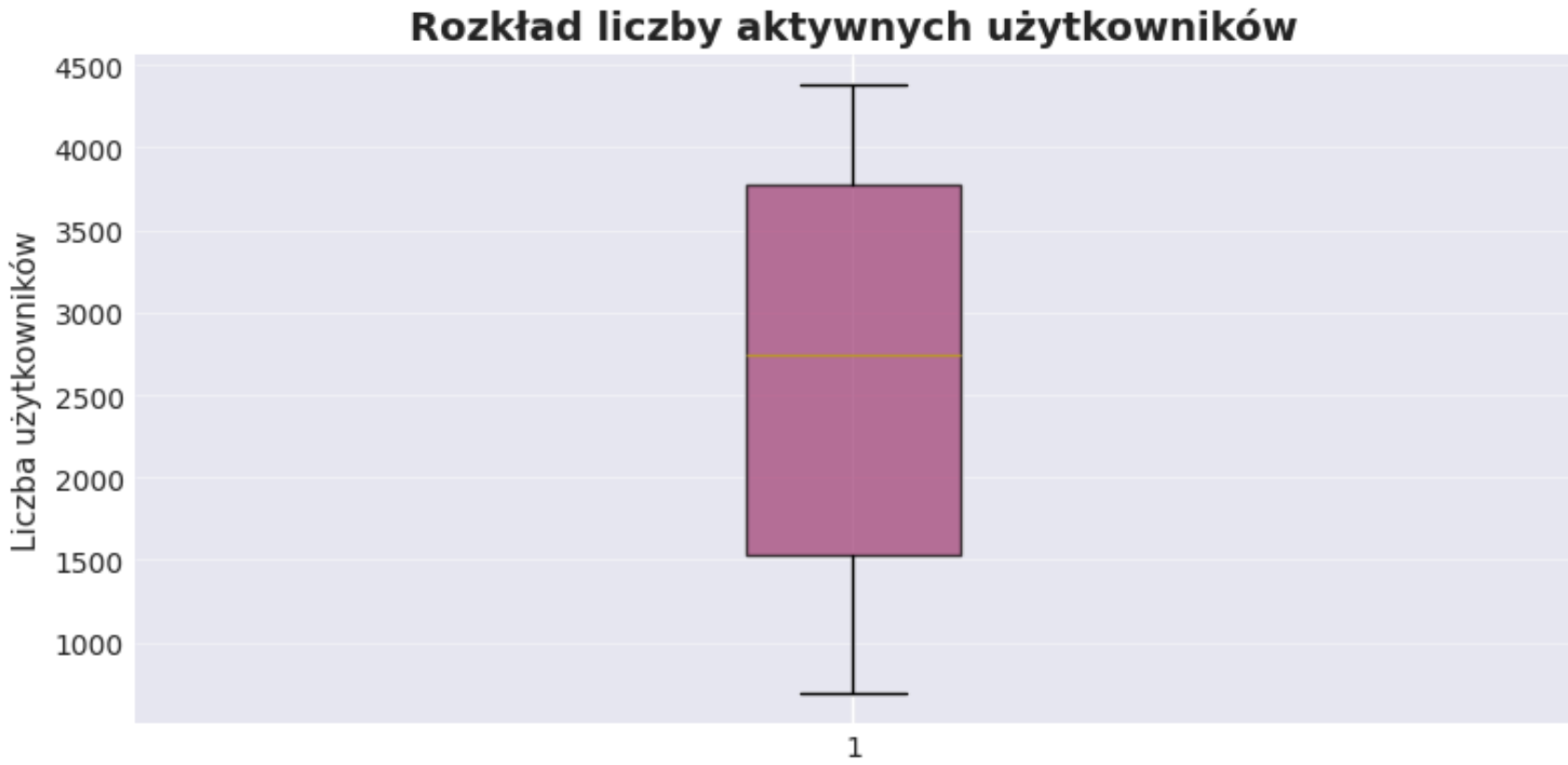
Wizualizacje analiz

Analizy w czasie rzeczywistym

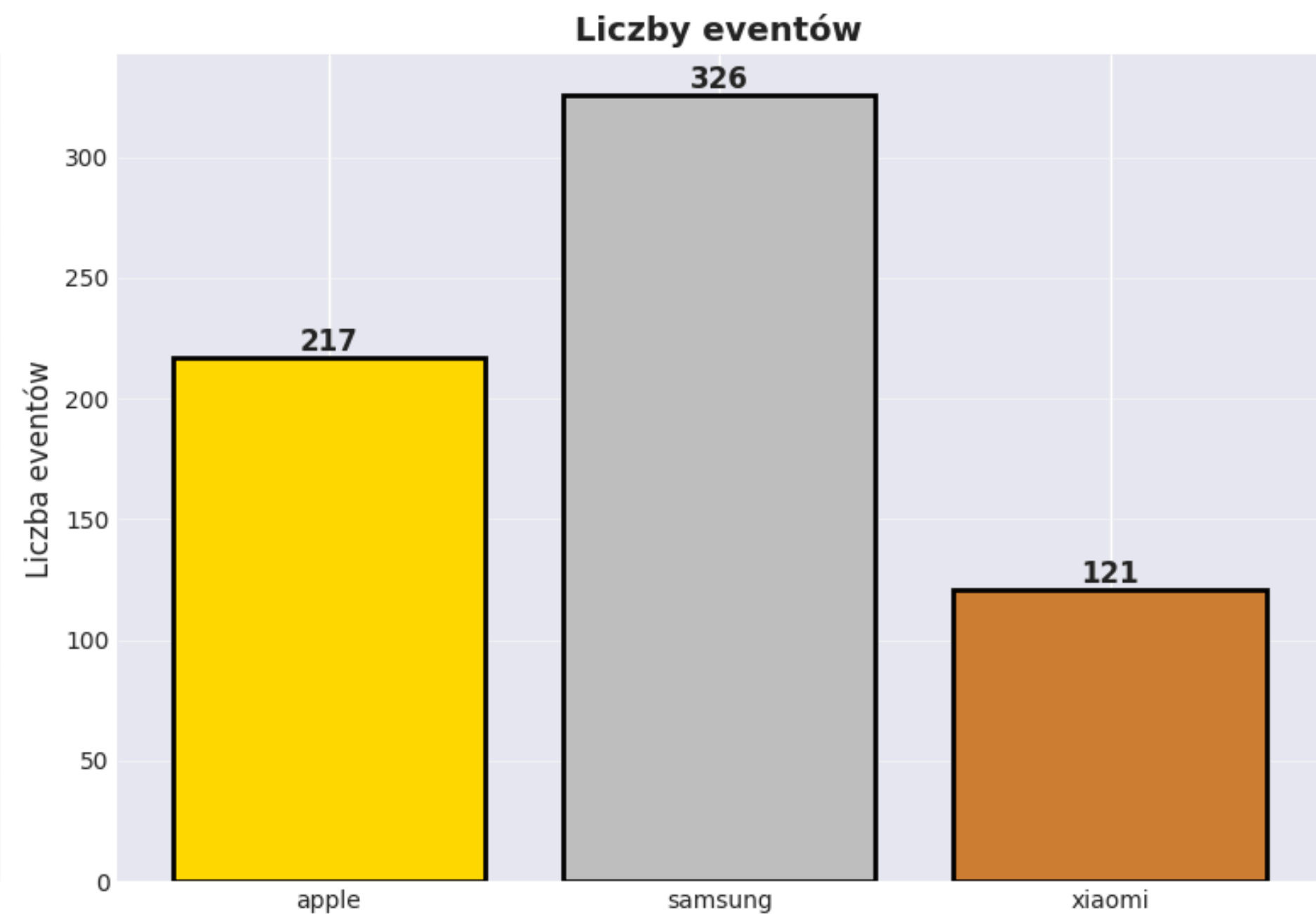
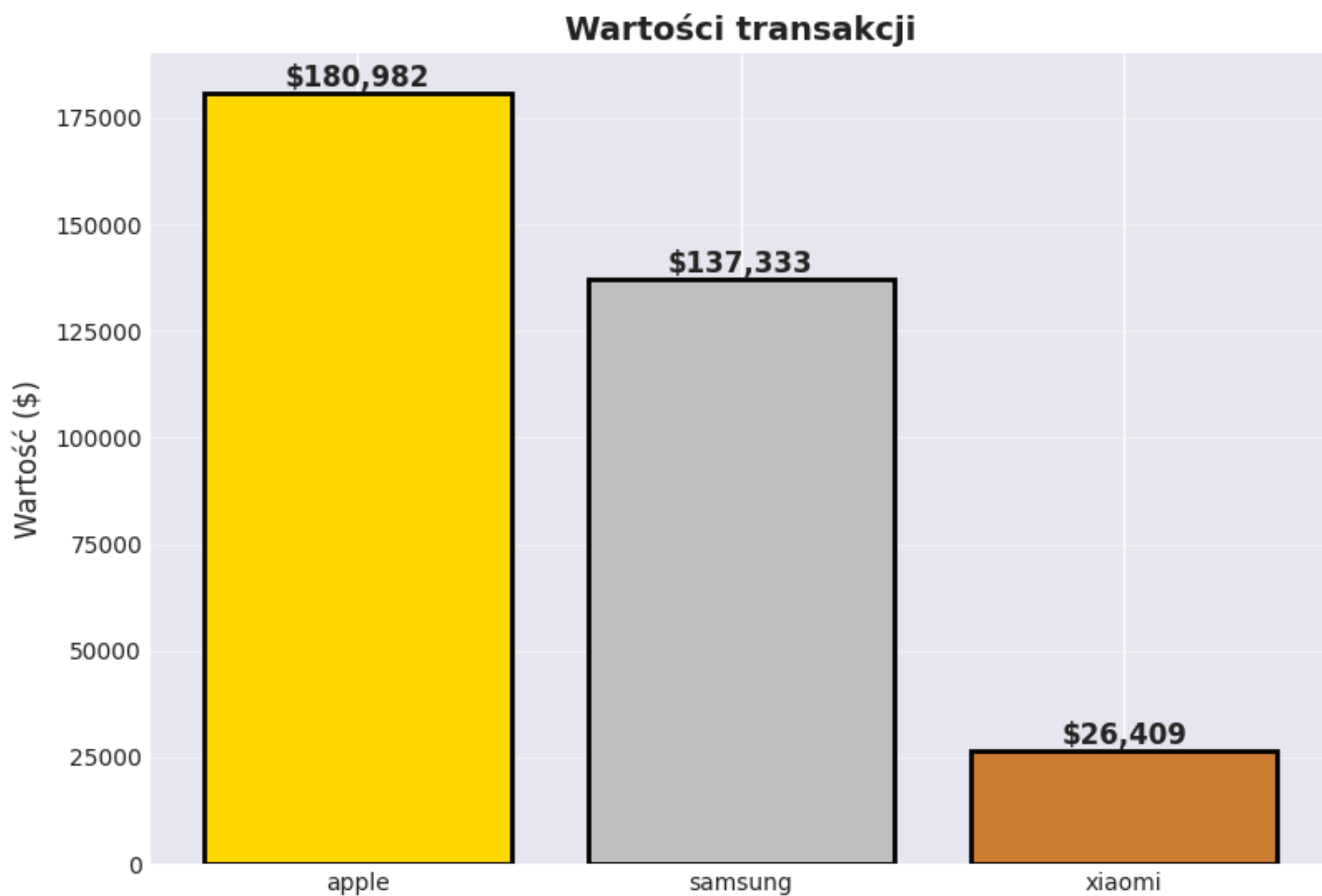
Analiza liczby aktywnych użytkowników (16:00 - 22:00, 30.12)



STATYSTYKI UŻYTKOWNIKÓW	
Średnia:	2654 użytkowników
Maksimum:	4380 użytkowników
Minimum:	690 użytkowników
Mediana:	2741 użytkowników
Odchylenie:	1180.3
Łącznie eventów:	585,365
Średnio eventów/user:	3.0



Top 3 Marki według



Top 3 marki

1. apple

Wartość: \$180,982.02

Eventy: 217

Średnia: \$834.02/event

2. samsung

Wartość: \$137,332.64

Eventy: 326

Średnia: \$421.27/event

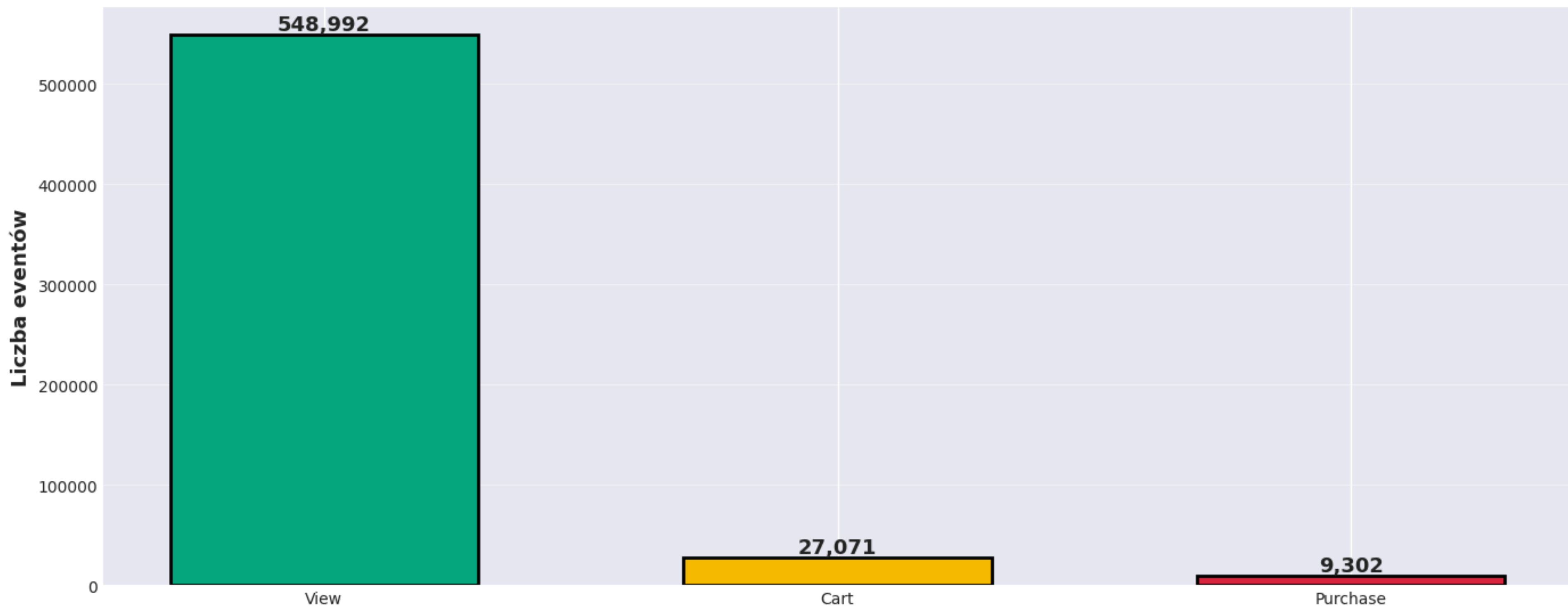
3. xiaomi

Wartość: \$26,408.56

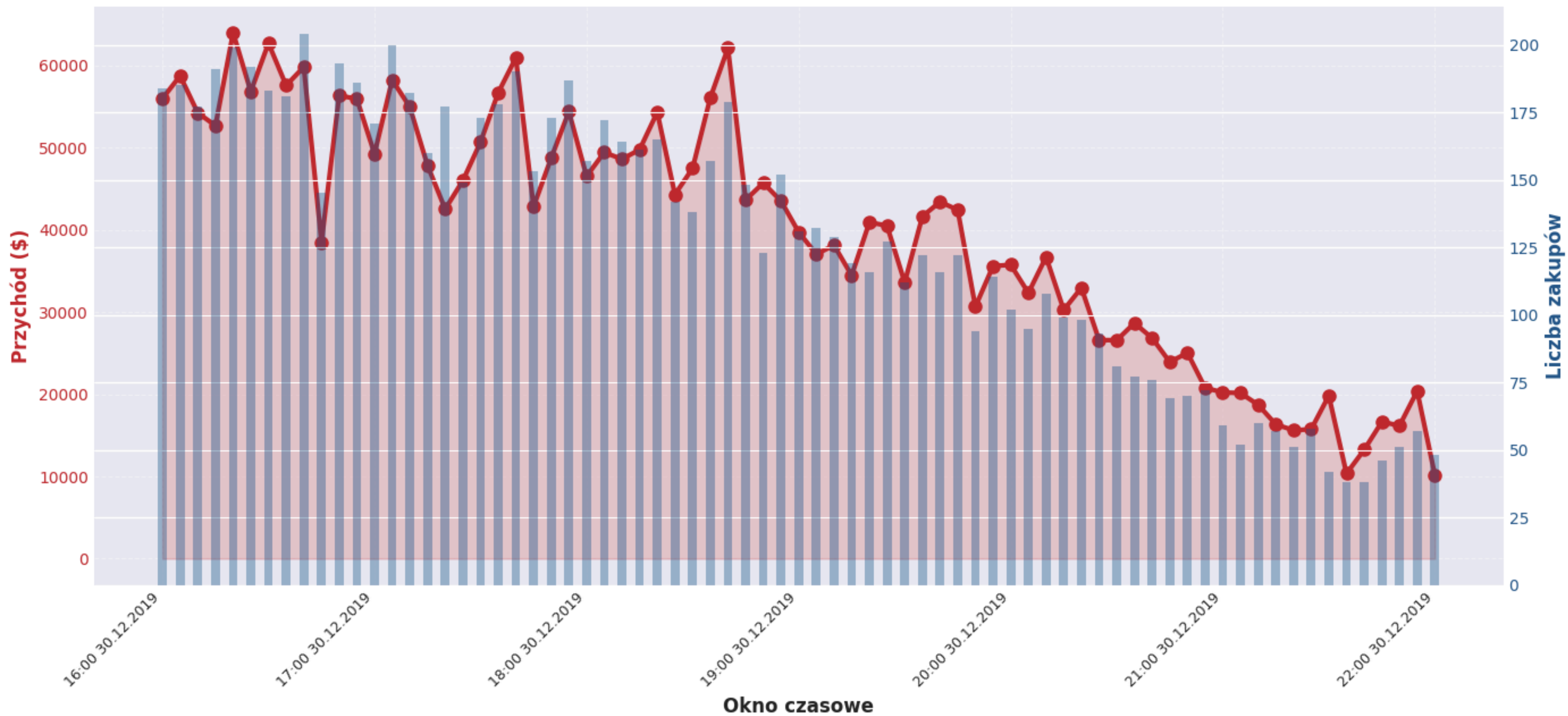
Eventy: 121

Średnia: \$218.25/event

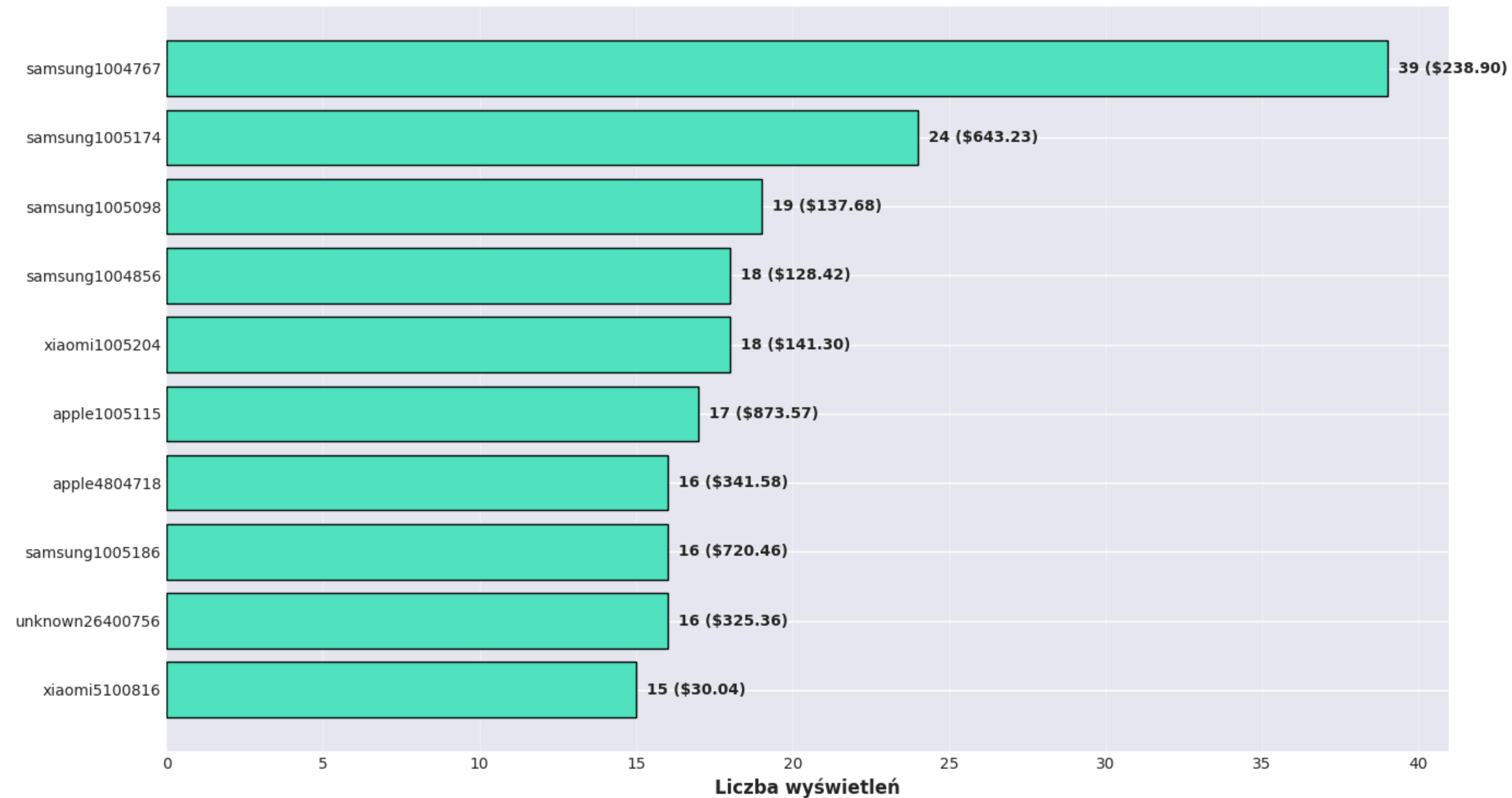
Lejek konwersji E-commerce



Przychody i zakupy w czasie



Top 10 Najczęściej Oglądanych Produktów



Podsumowanie



Analizowanych okien czasowych: **73**



Średnia liczba aktywnych użytkowników: **2654**



Łączna liczba eventów: **585,365**



Najpopularniejsza marka: **apple (\$180,982.02)**

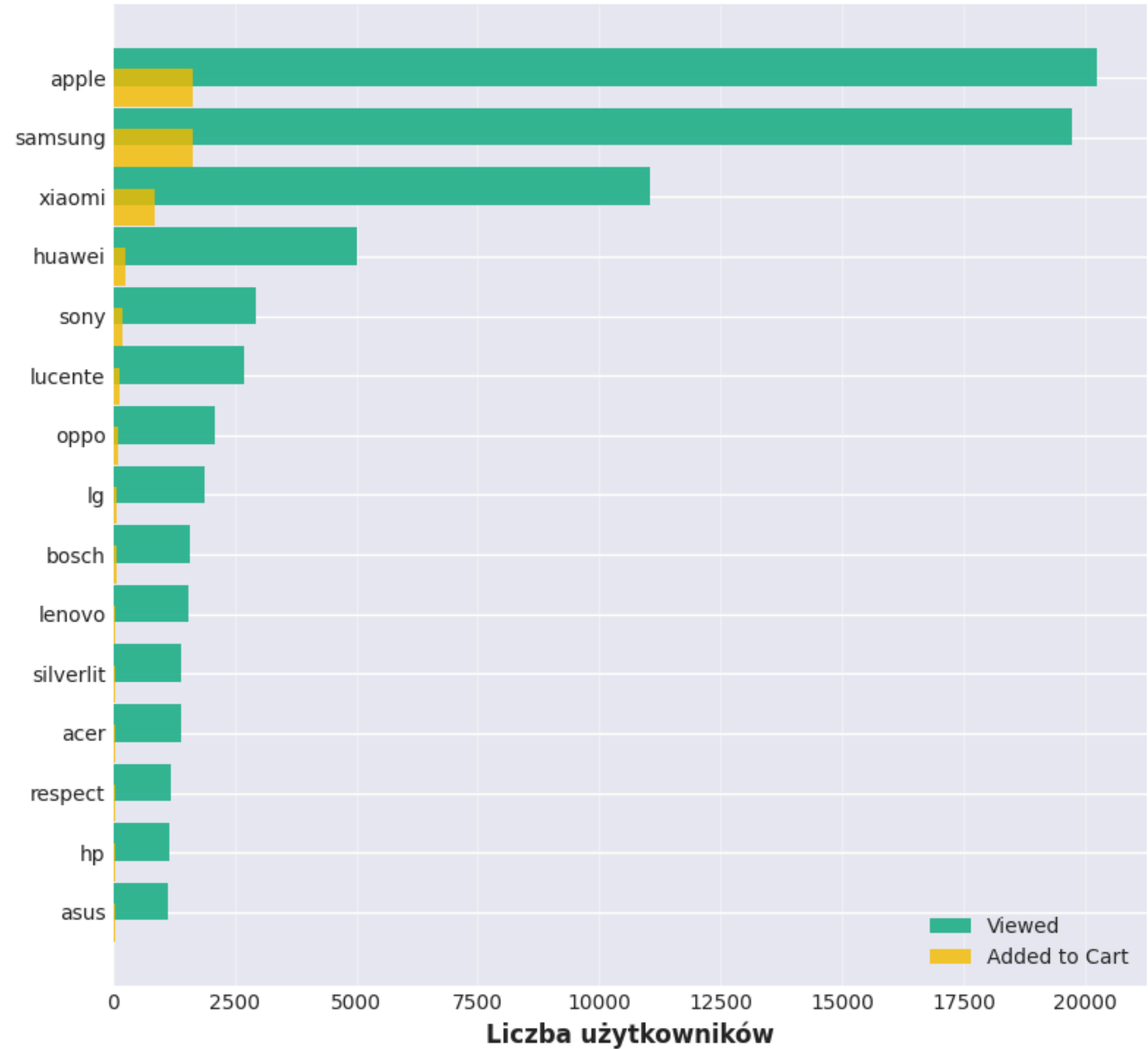


Najczęściej oglądany produkt: **samsung**

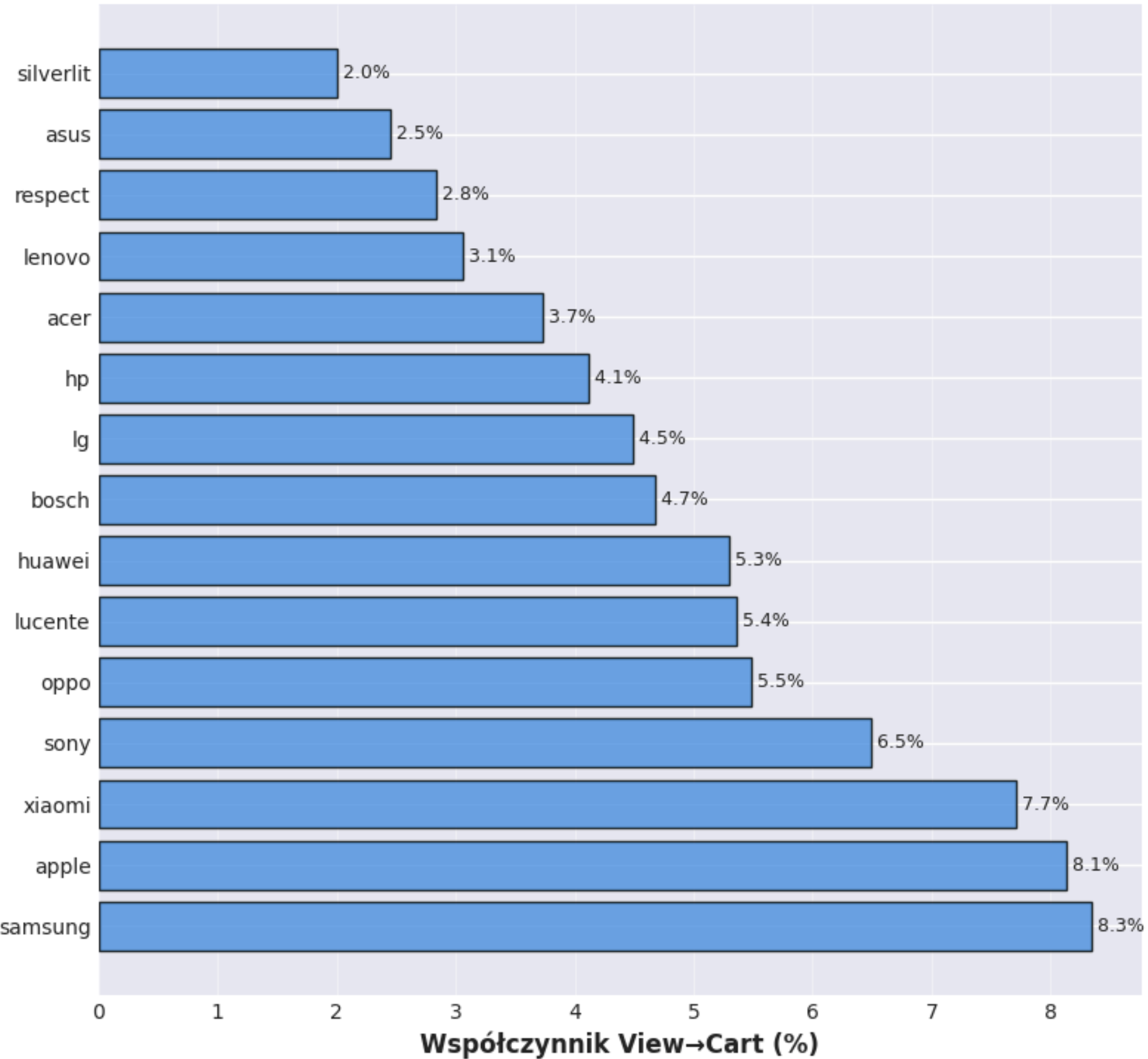
Analiza lejka zakupowego

Top 15 Marek - Konwersja View → Add to Cart

Użytkownicy: Viewed vs Cart

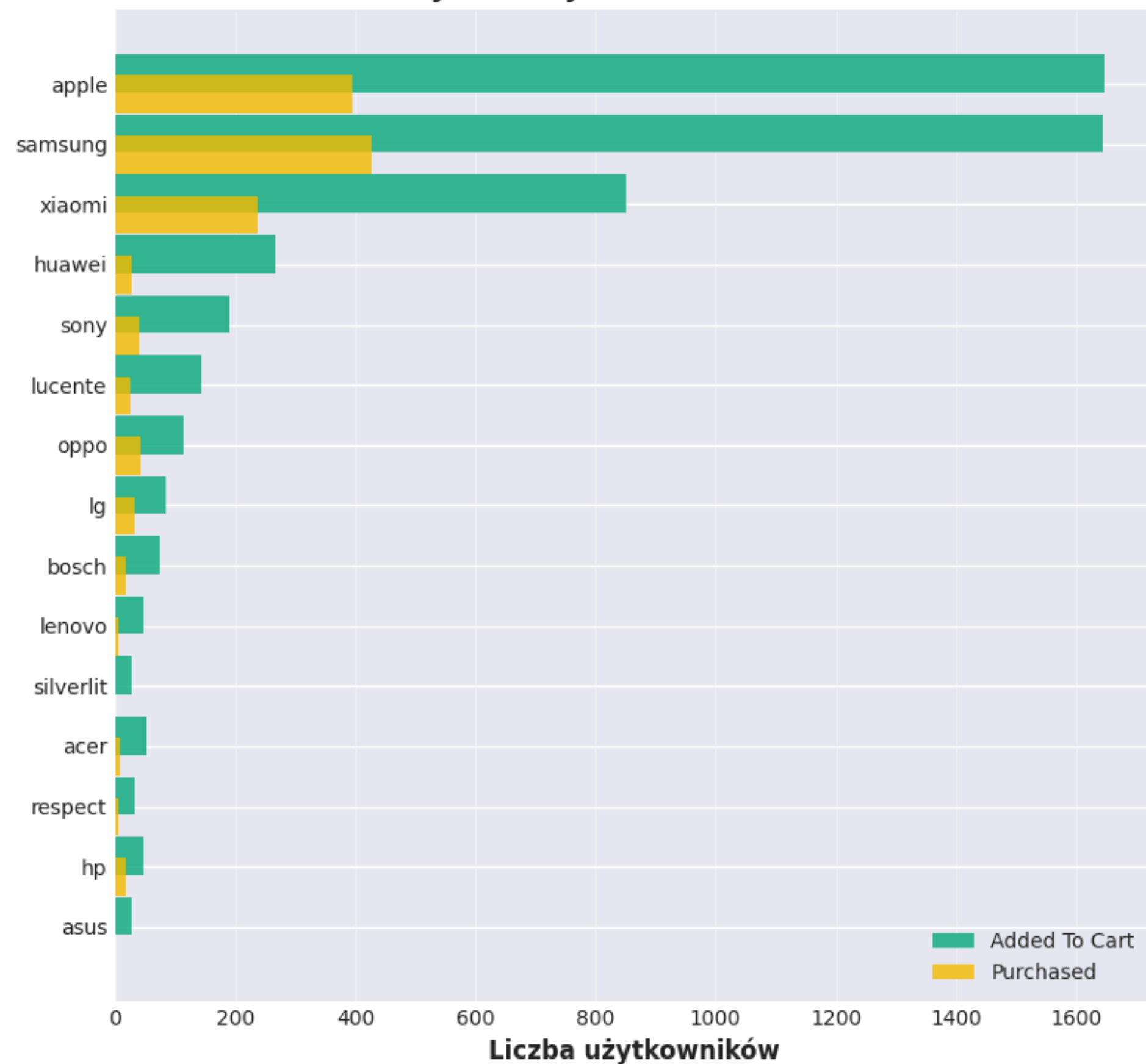


Współczynnik konwersji

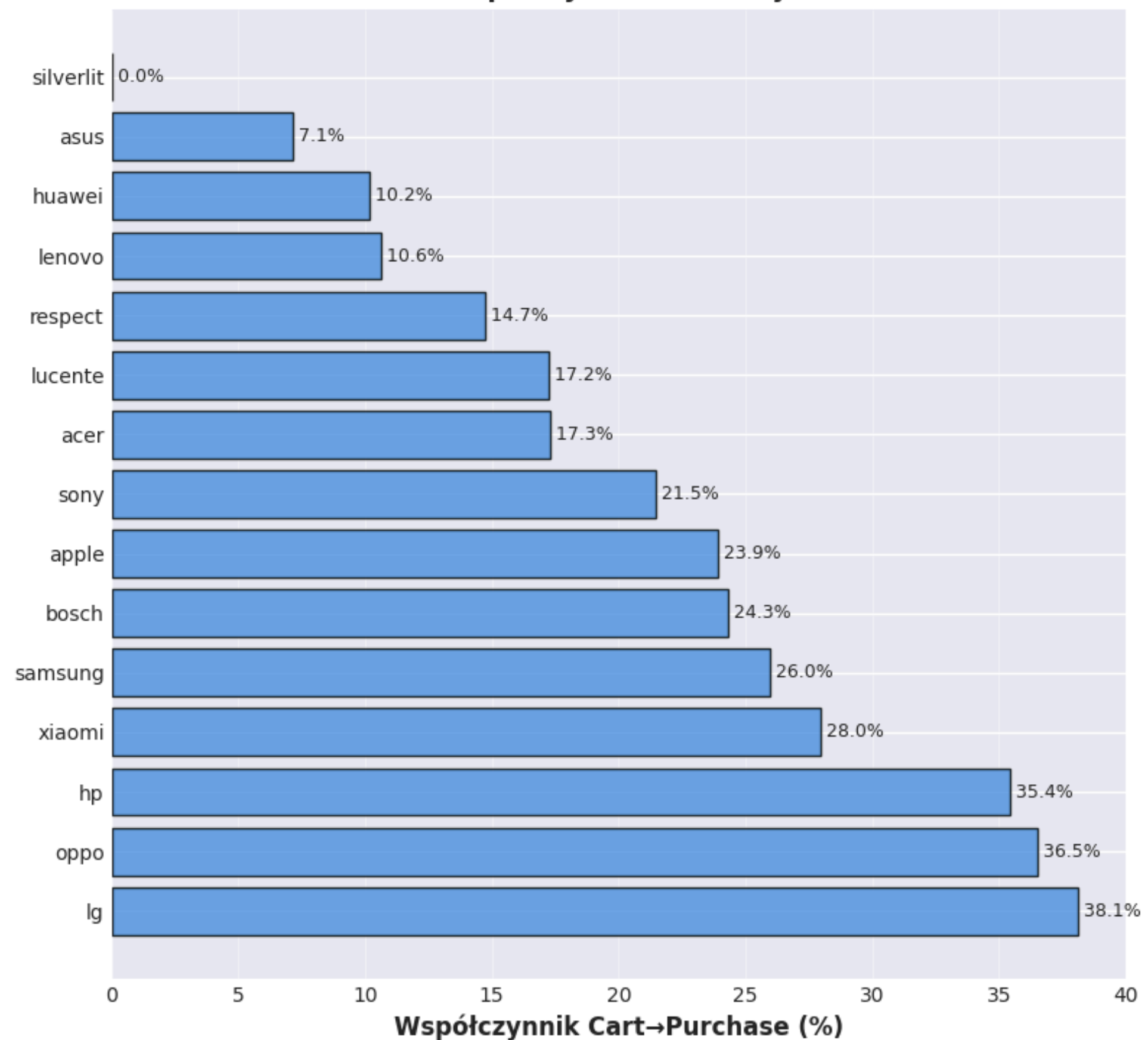


Top 15 Marek - Konwersja Add to Cart → Purchase

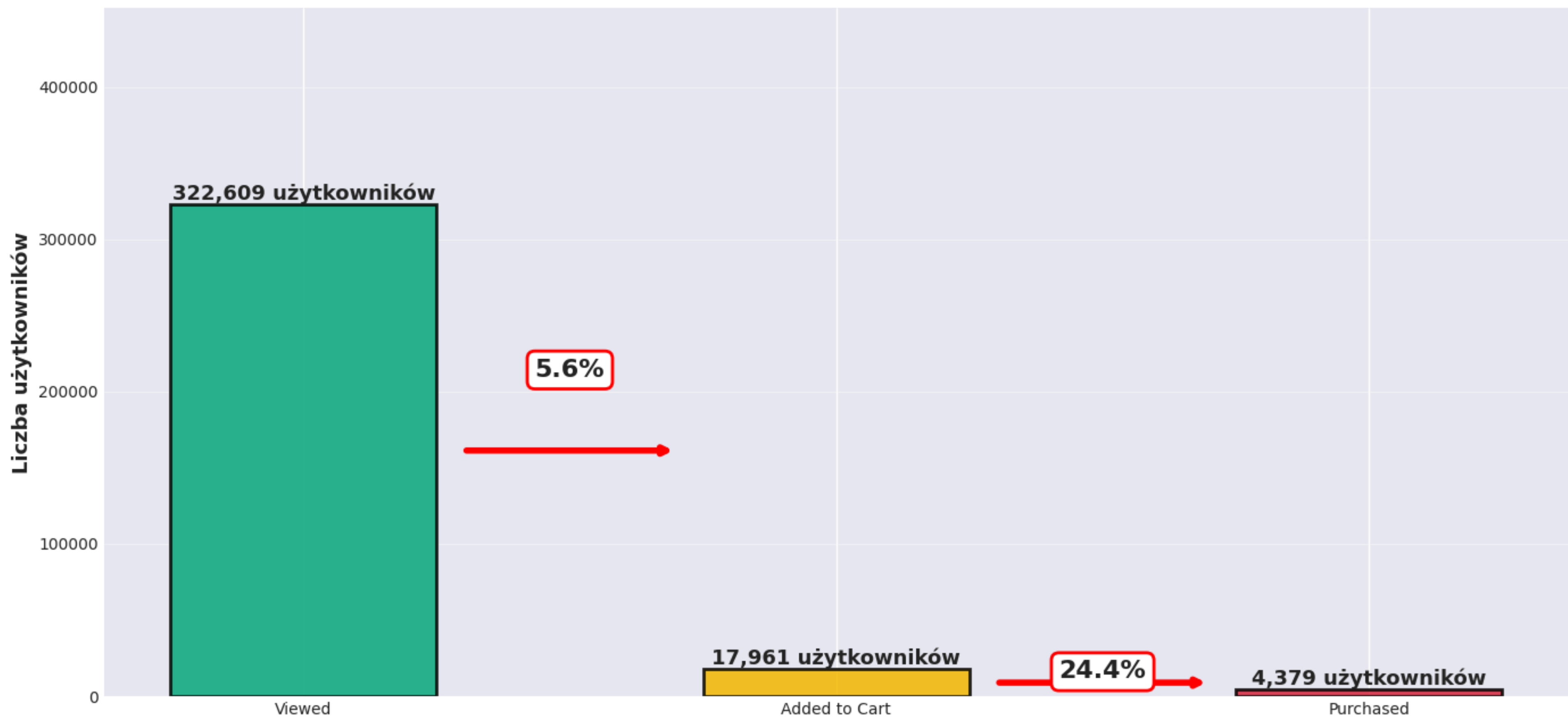
Użytkownicy: Cart vs Purchased



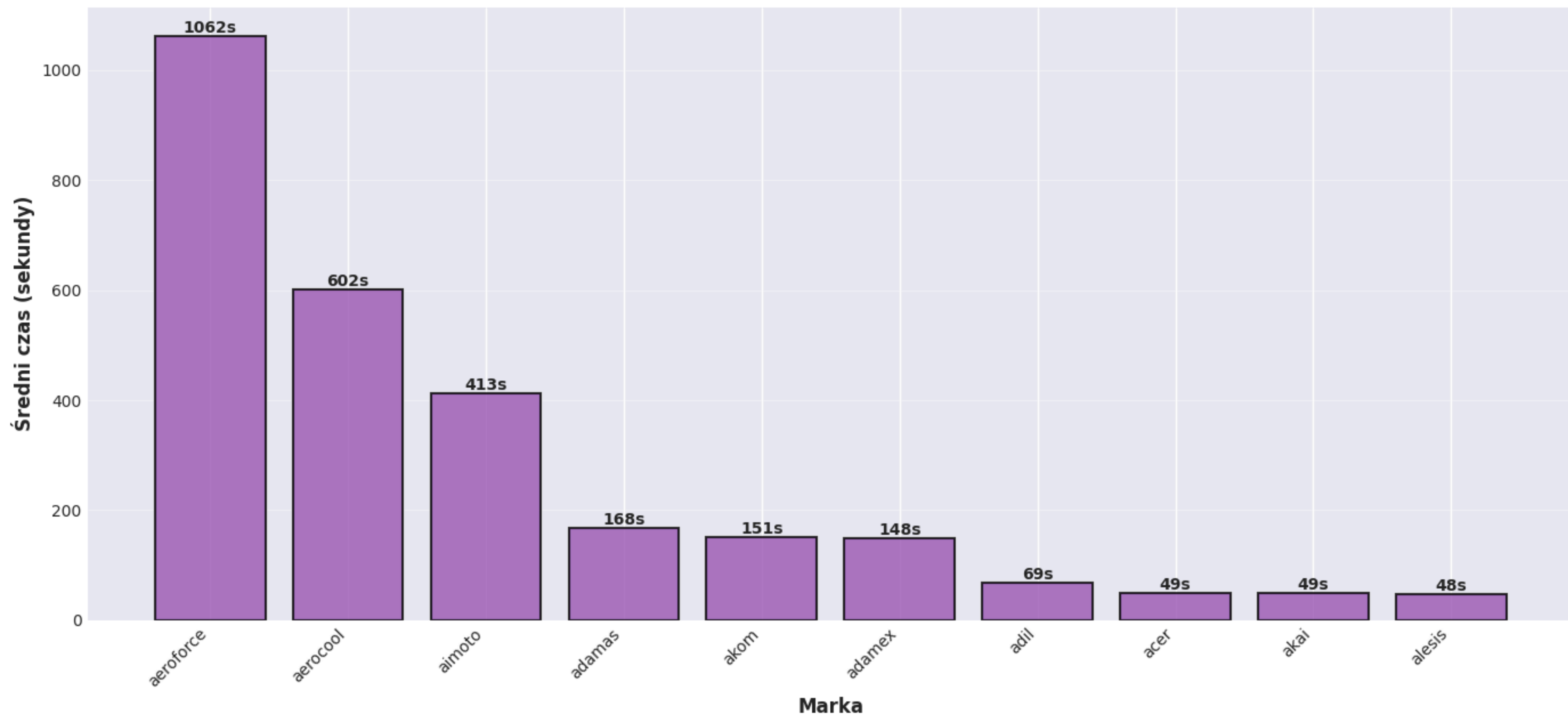
Współczynnik konwersji



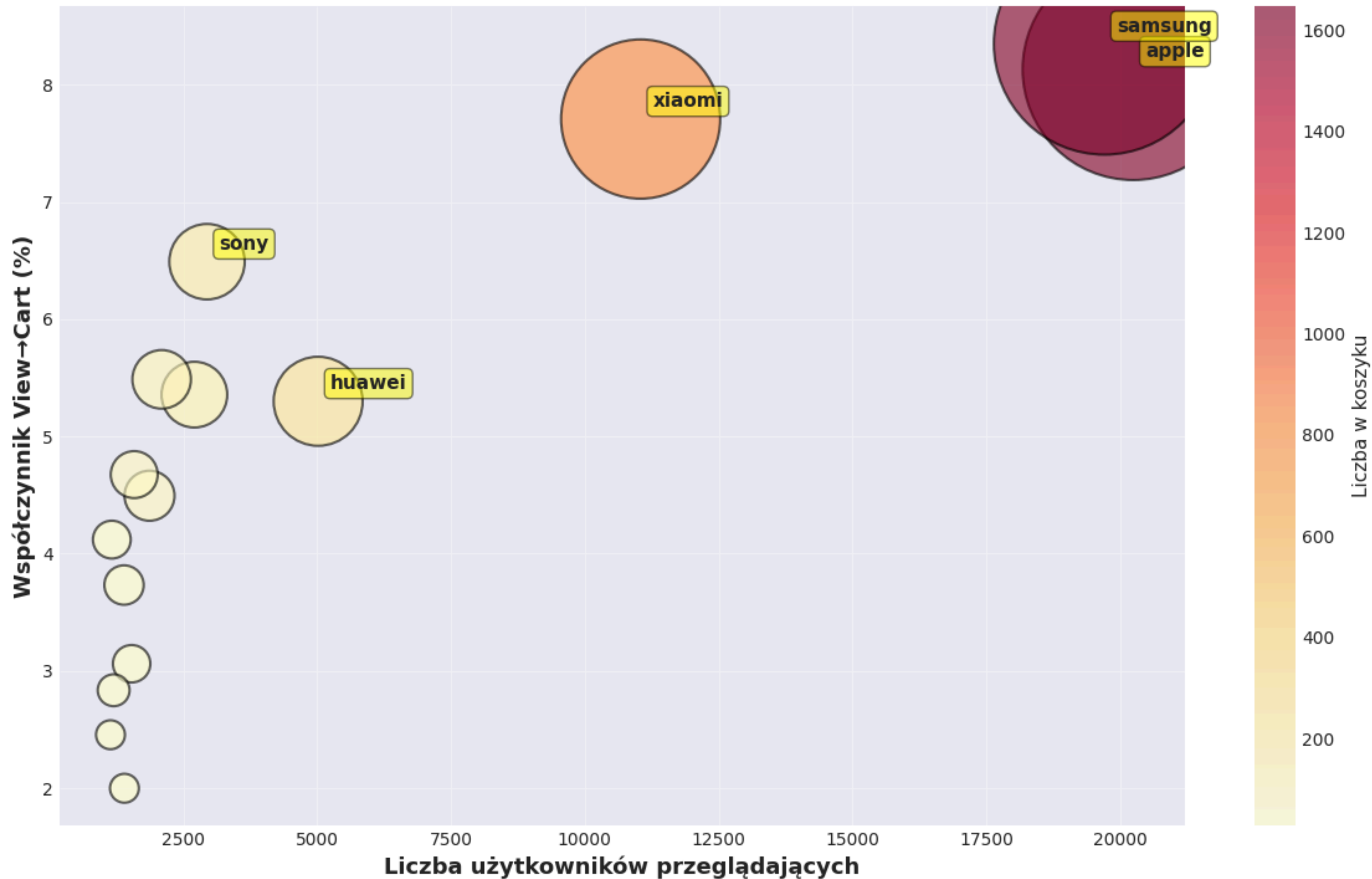
Całociowy lejek konwersji (wszystkie marki)



Średni Czas View → Add to Cart (Top 10 Marek)

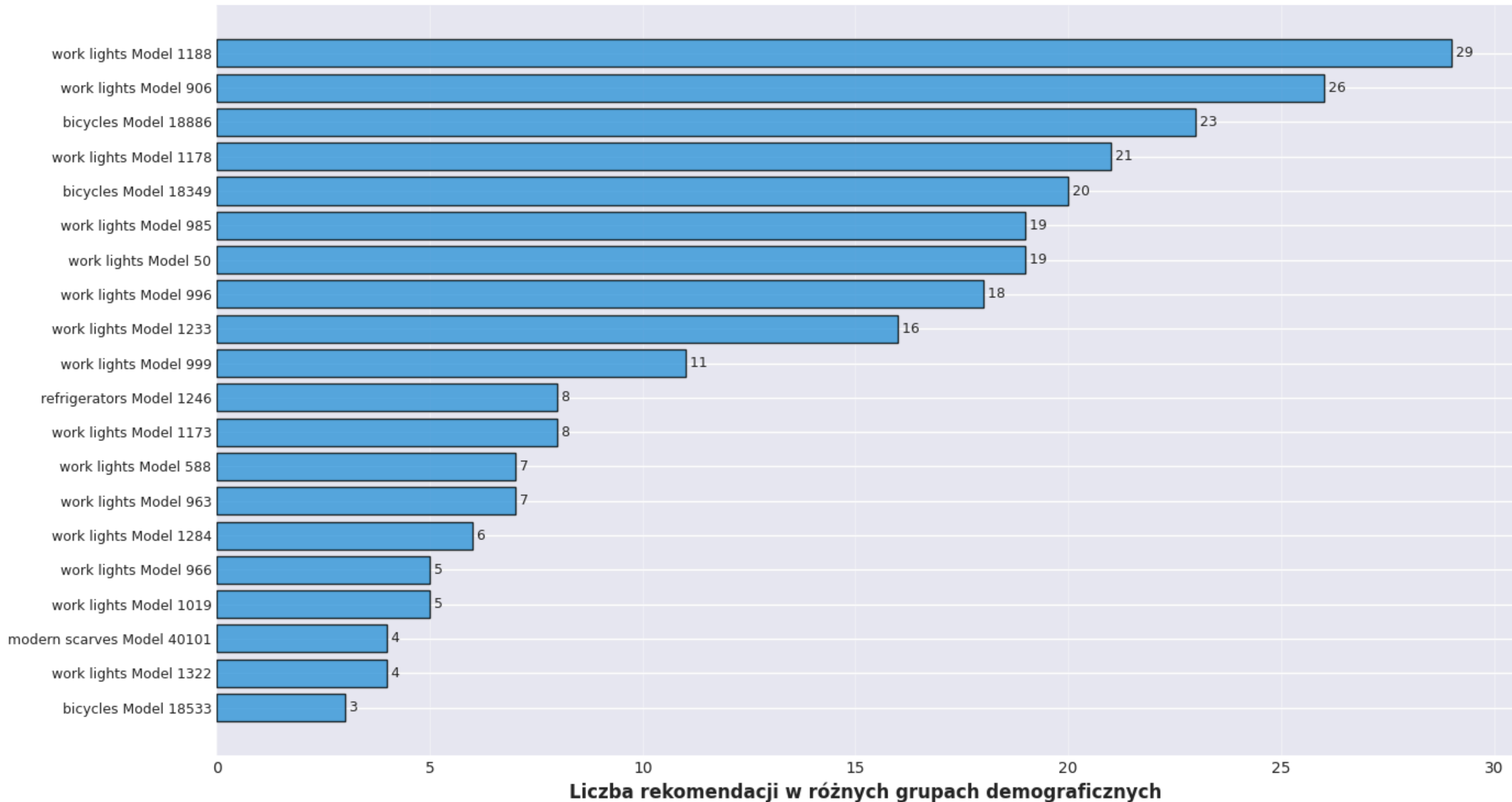


Popularność vs Konwersja
(rozmiar = liczba dodanych do koszyka)

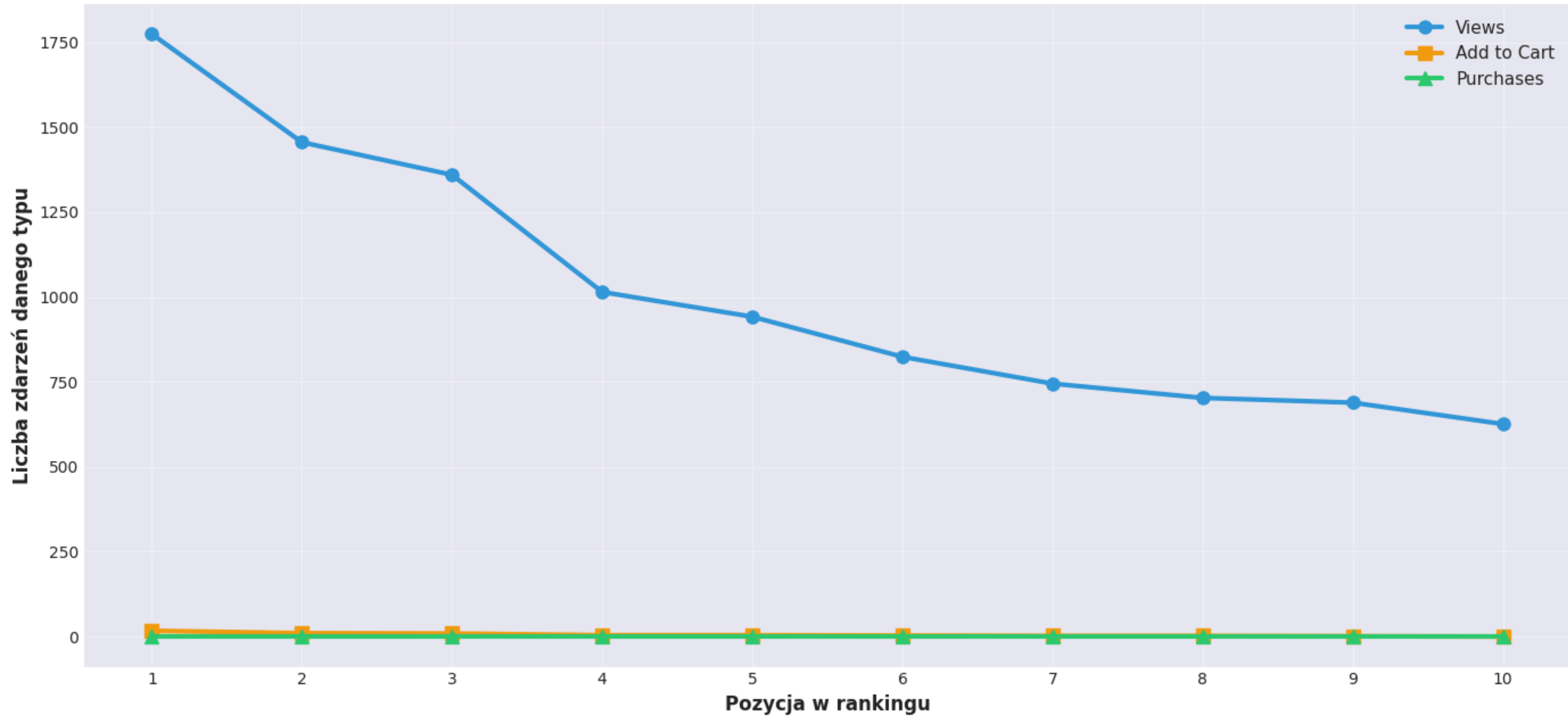


Analiza demograficzna

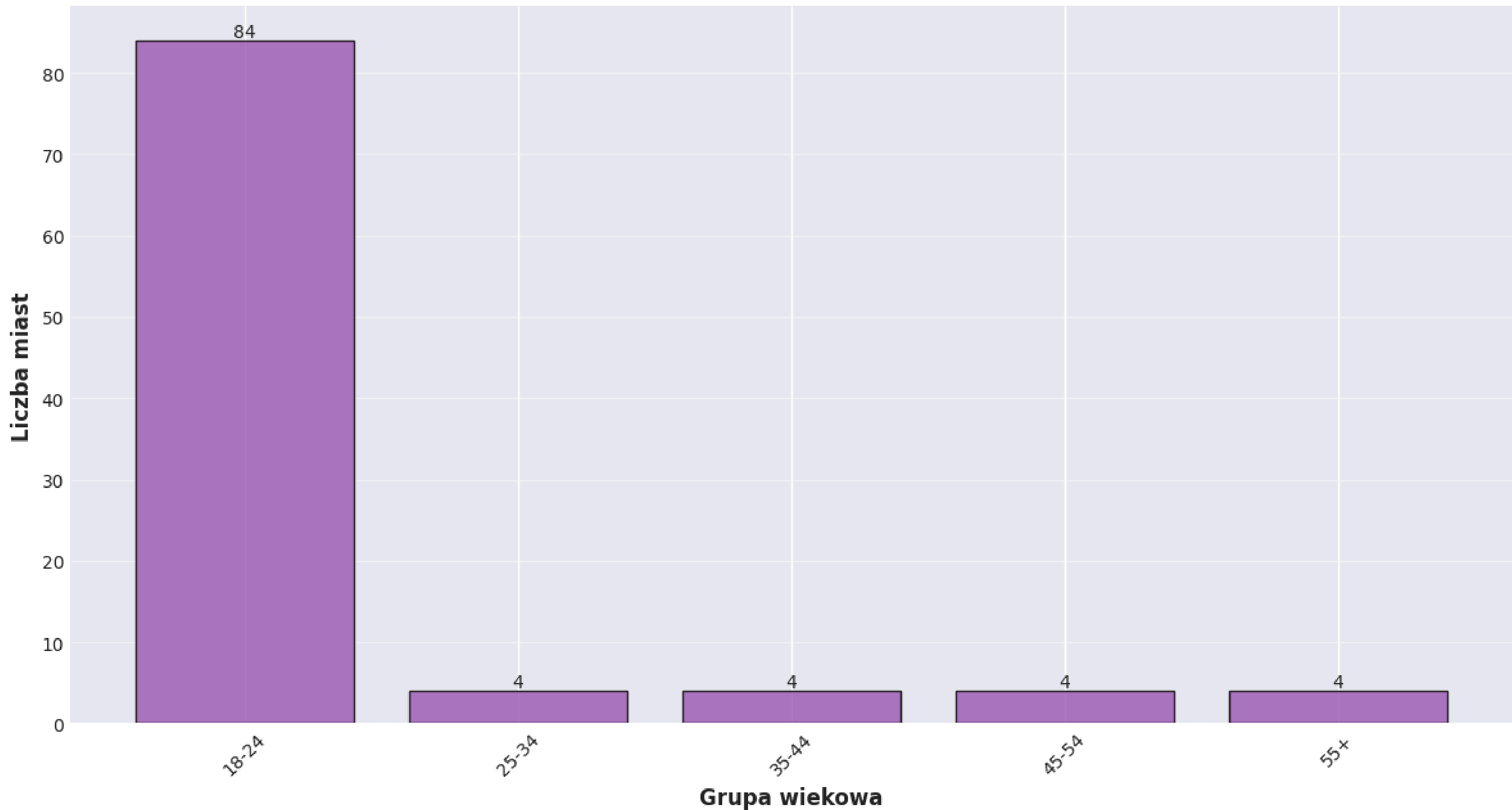
Top 20 Najczęściej Rekomendowanych Produktów



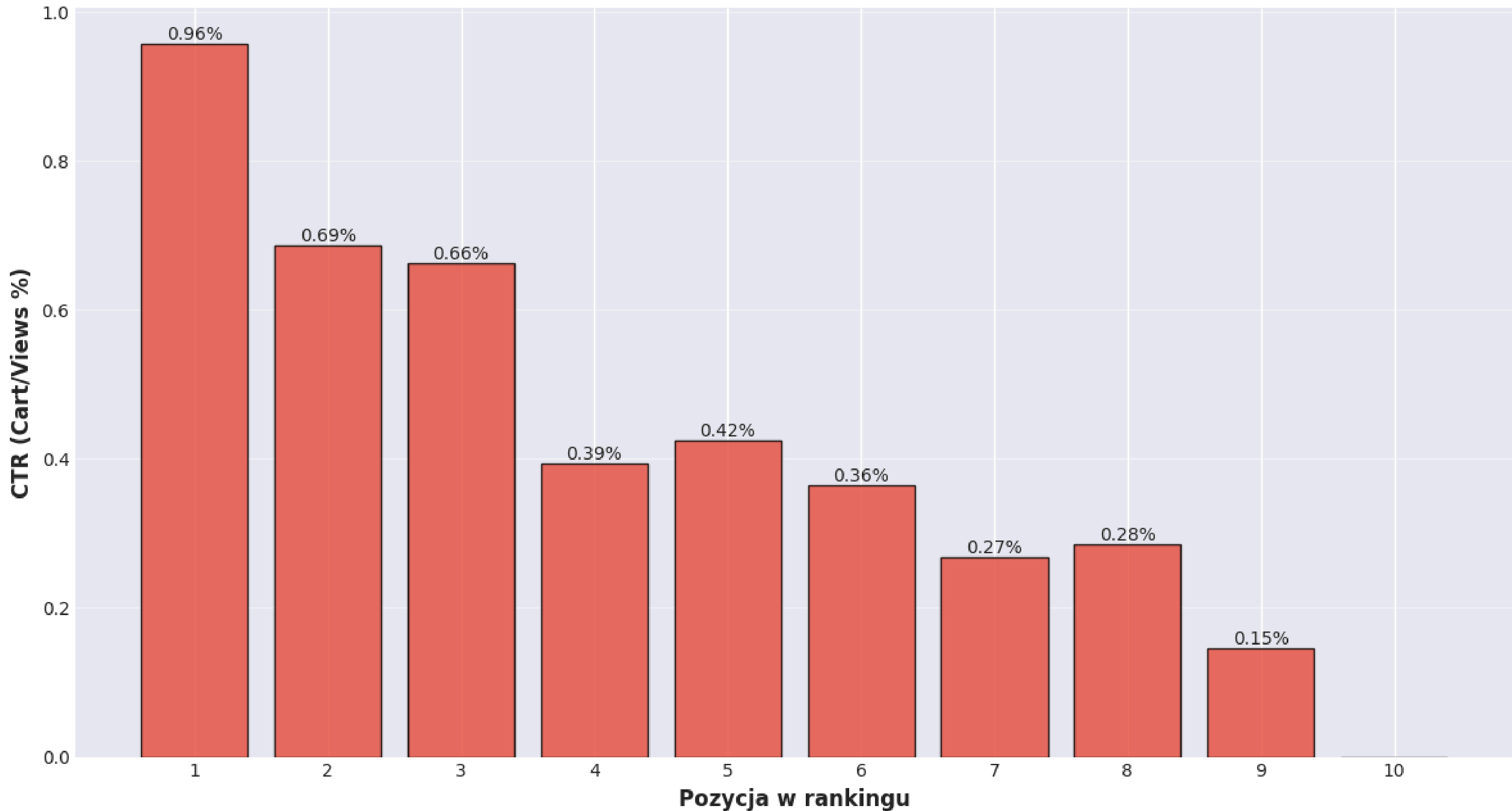
Liczba zdarzeń według pozycji w rankingu



Liczba miast, z których pochodzą reprezentanci grup wiekowych



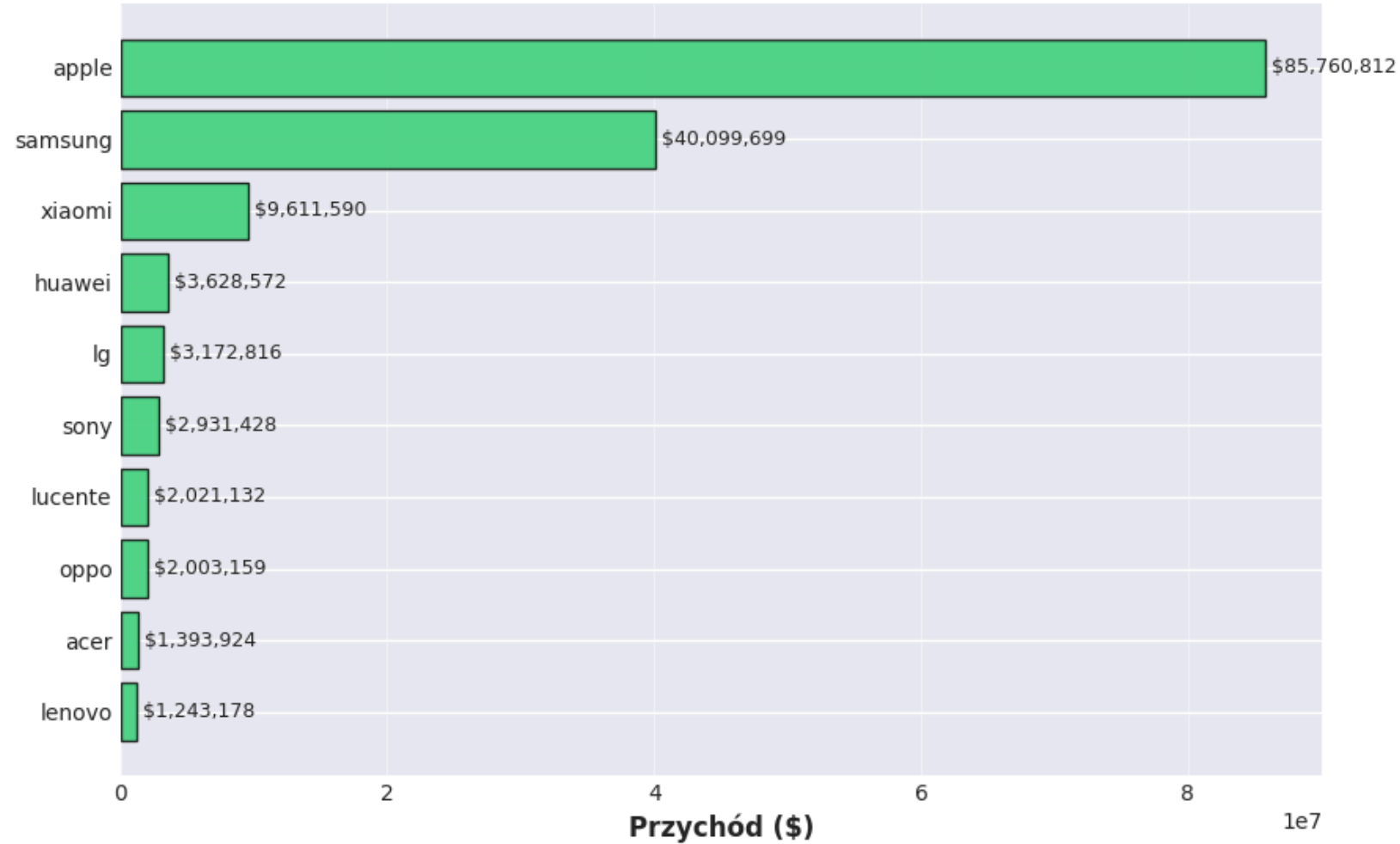
Click-Through Rate według pozycji w rankingu



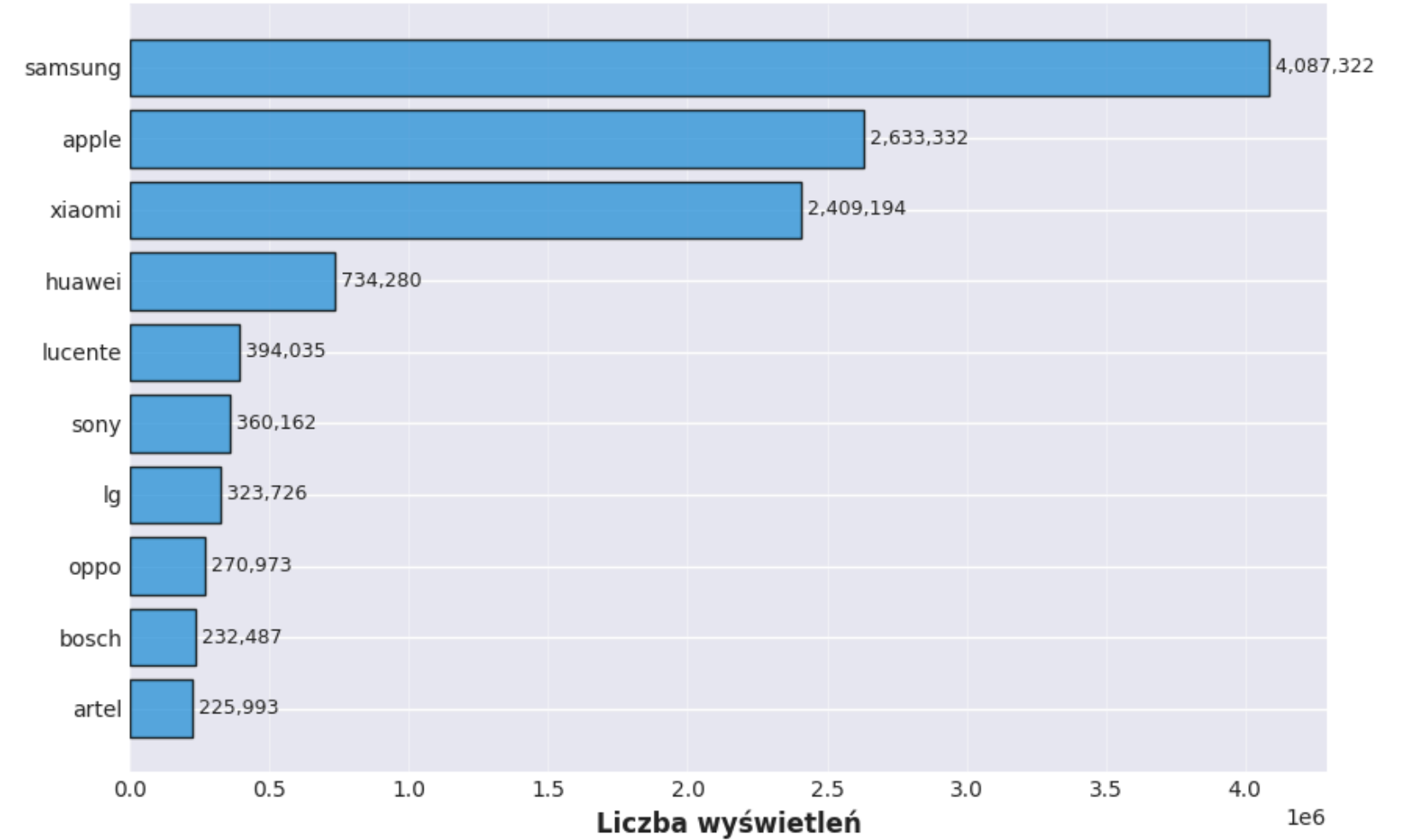
Analiza marek

Analiza Top 10 Marek

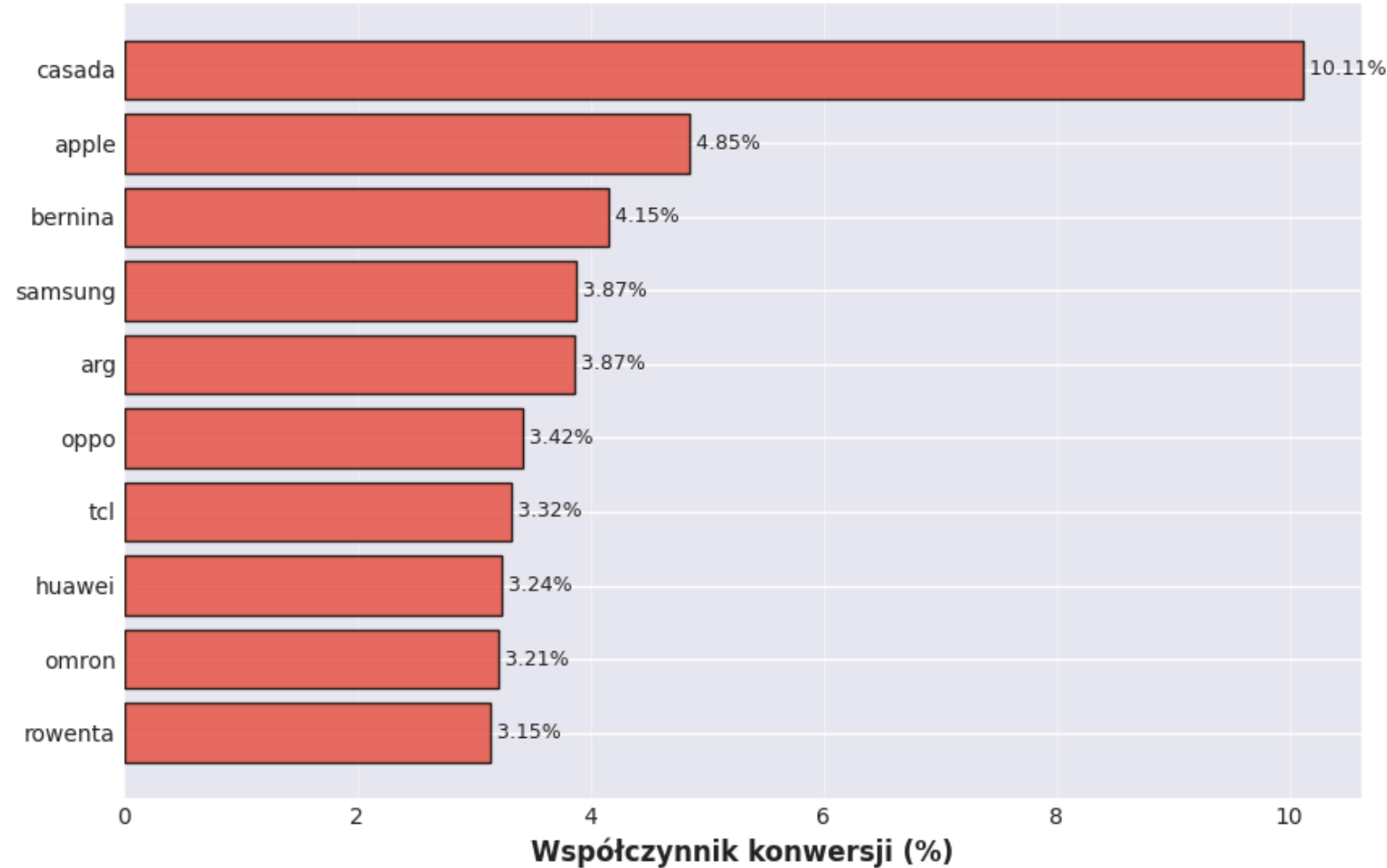
Top 10 według przychodu



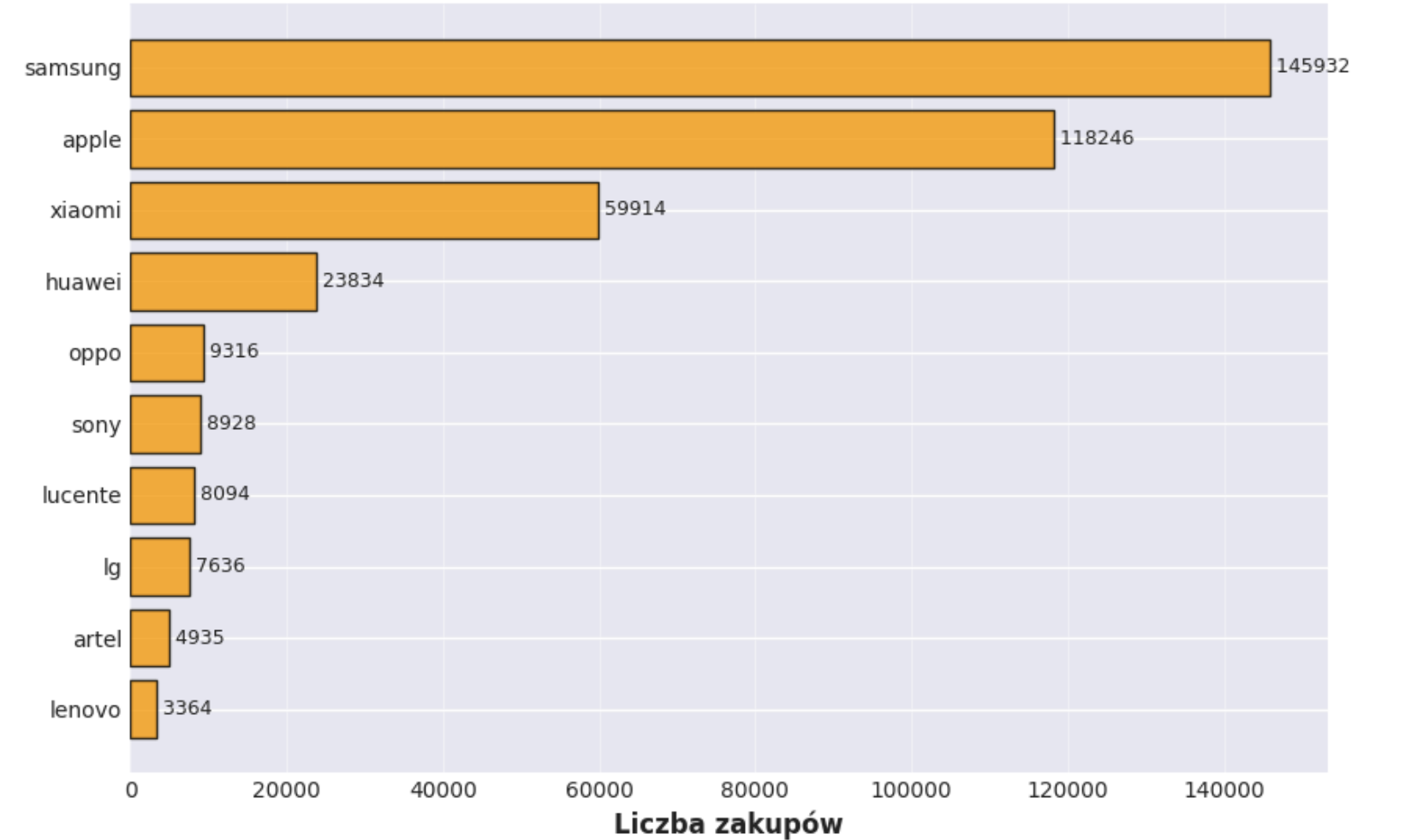
Top 10 według liczby wyświetleń



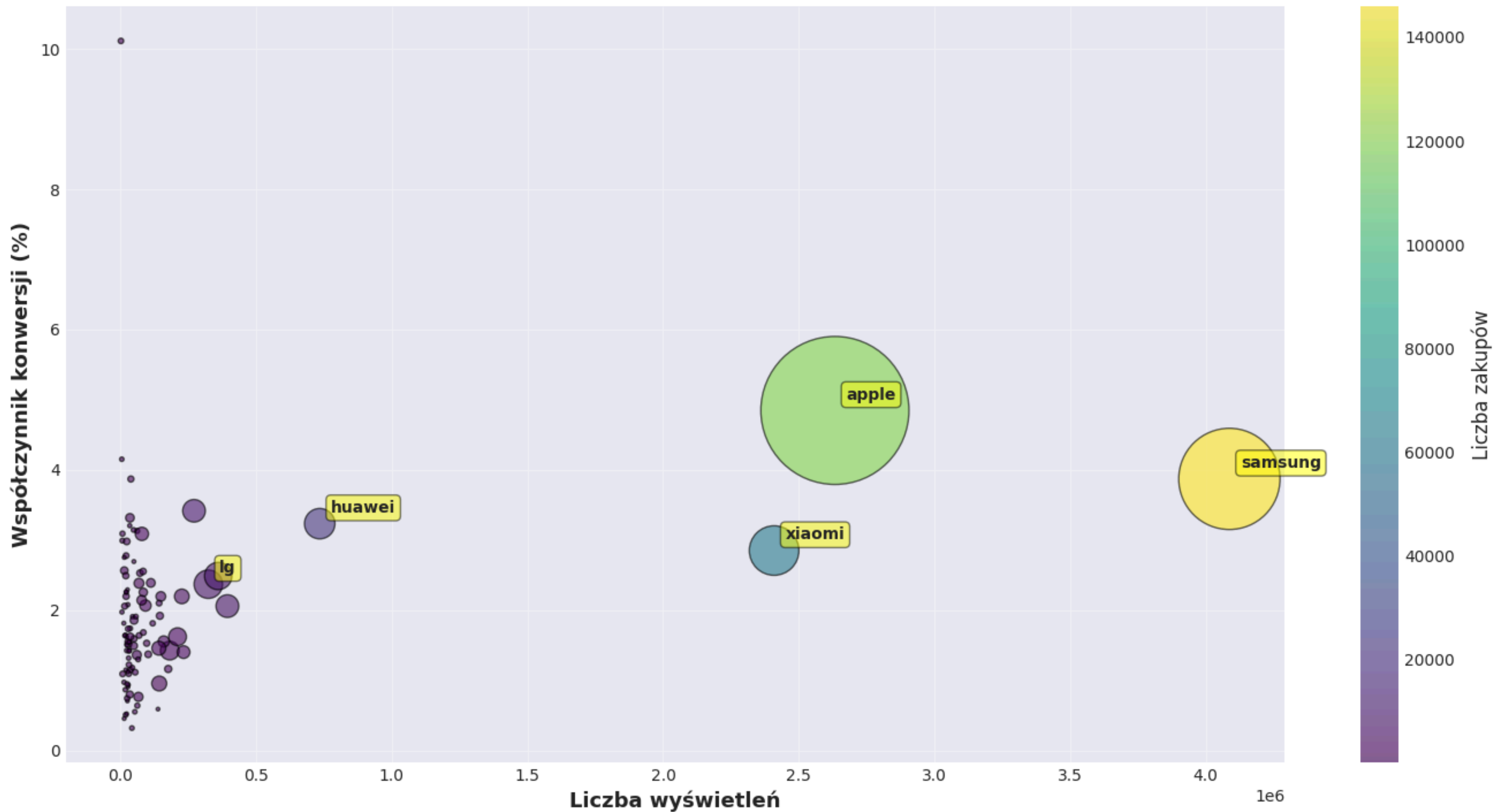
Top 10 według konwersji



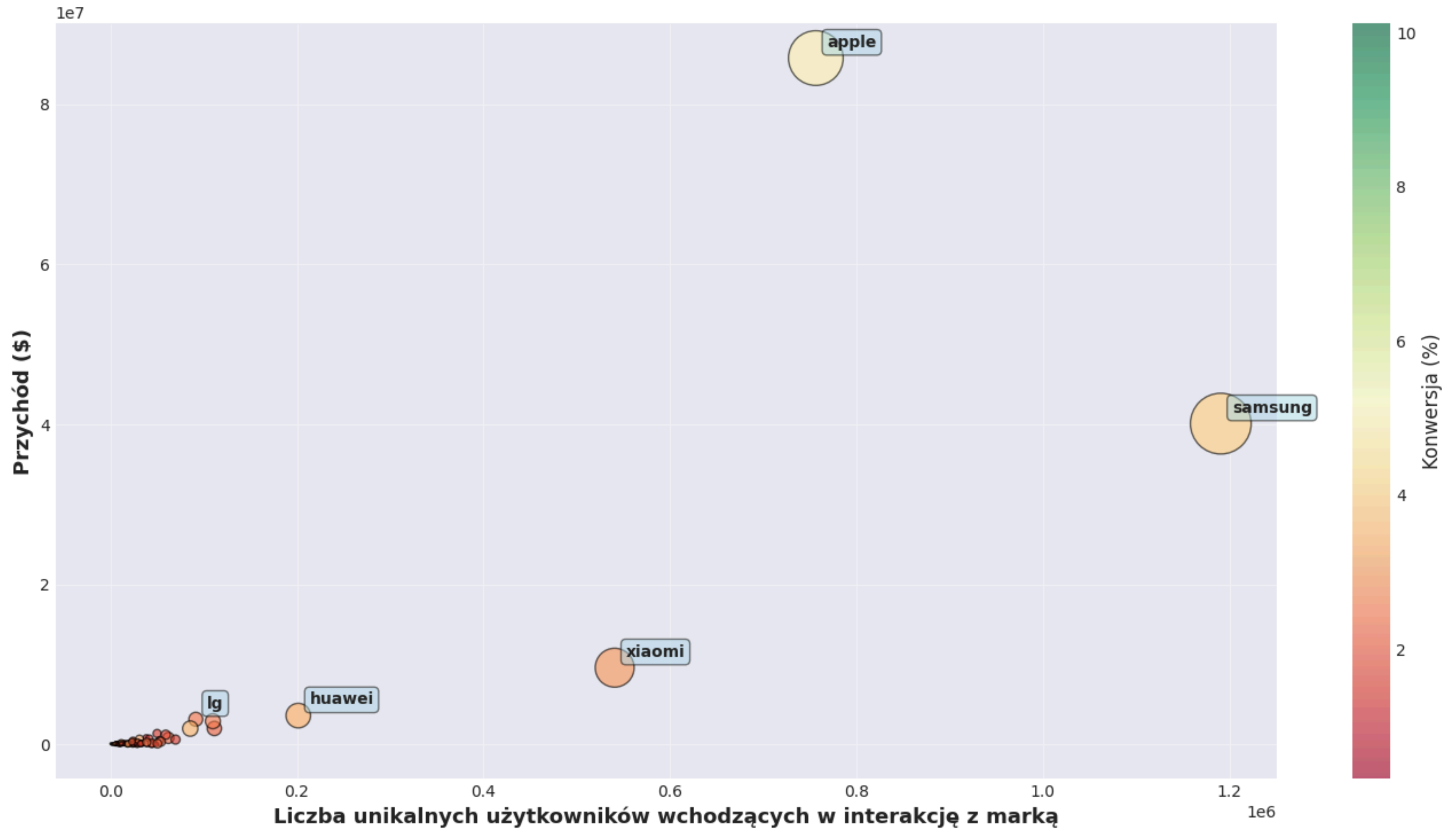
Top 10 według zakupów



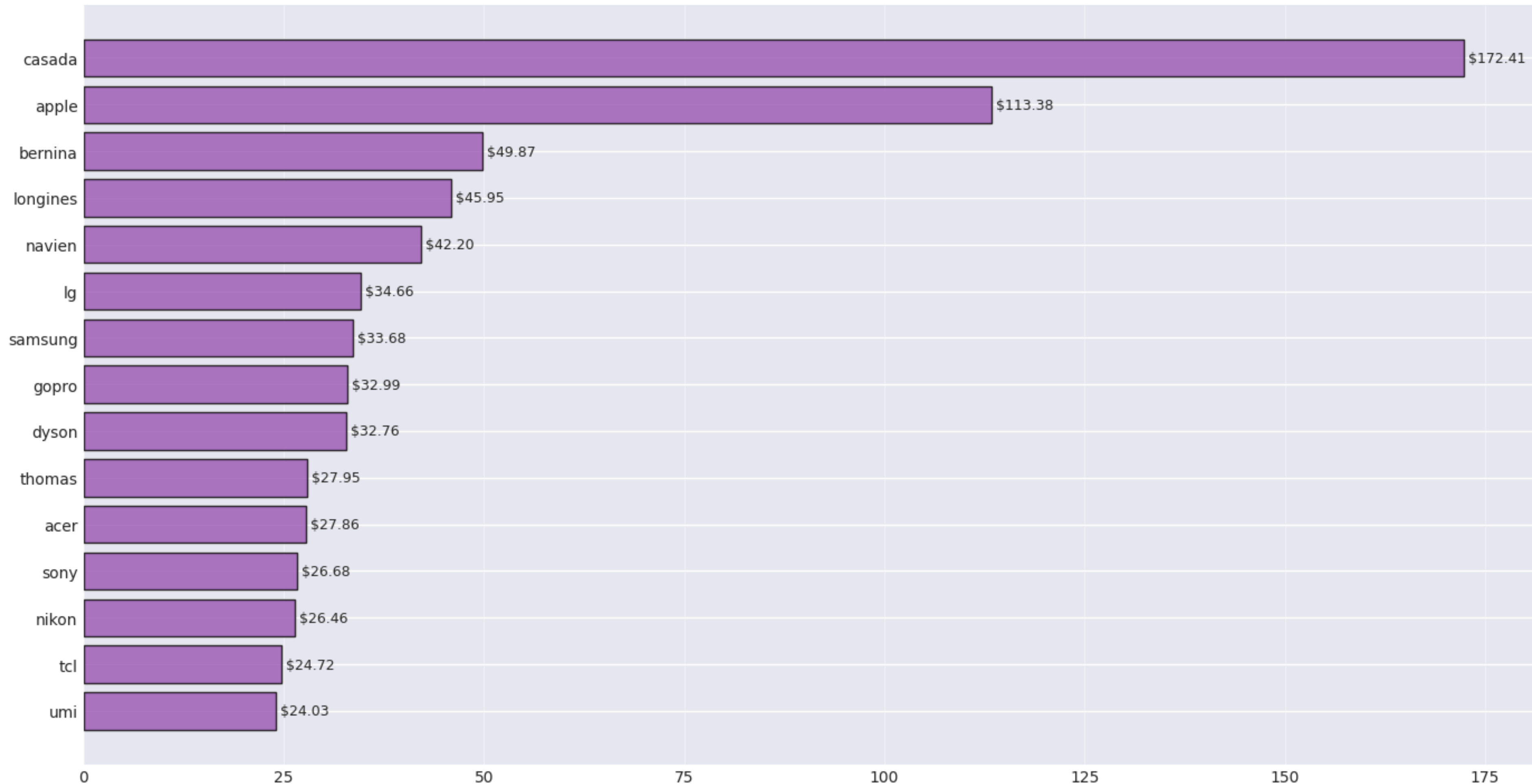
Wyświetlenia vs Konwersja
(rozmiar bąbelka = przychód, kolor = liczba zakupów)



Użytkownicy vs Przychód (rozmiar = zakupy, kolor = konwersja %)



Top 15 Marek według ARPU



ARPU - Średni przychód na użytkownika wchodzącego w interakcję z marką (\$)

Statystyki analizy marek

Ogólne:

- Liczba marek: 100
- Łączny przychód: \$167,289,847.31
- Łączne wyświetlenia: 16,408,782
- Łączne zakupy: 470919
- Średnia konwersja: 1.95%

Top 3 marki według przychodu:

1. apple: \$85,760,812.24 (118246 zakupów, 4.85% konwersja)
2. samsung: \$40,099,698.56 (145932 zakupów, 3.87% konwersja)
3. xiaomi: \$9,611,589.74 (59914 zakupów, 2.85% konwersja)

Top 3 marki według konwersji:

1. casada: 10.11% (1628 views, 174 zakupów)
2. apple: 4.85% (2633332 views, 118246 zakupów)
3. bernina: 4.15% (4716 views, 177 zakupów)

Top 3 marki według ARPU:

1. casada: \$172.41/user (677 użytkowników)
2. apple: \$113.38/user (756380 użytkowników)
3. bernina: \$49.87/user (1524 użytkowników)

Insights:

- Marek z wysokim ruchem (>Q3): 25
- Średnia konwersja top quartile: 2.12%
- Najwyższe ARPU: casada (\$172.41)

7 stycznia 2026 r.

Dziękujemy za uwagę!

Zespół KGW Gawron

Natalia Choszczyk, Filip Langiewicz, Mikołaj Rowicki

